

Lec.1

Artificial Intelligence

What is Artificial Intelligence?

[Def. 1] **Artificial intelligence (AI) may be defined as the branch of computer science that is concerned with the automation of intelligent behavior.** This definition is particularly appropriate in that it emphasizes a conviction that **AI is a part of computer science** and, as such, **must be based on sound theoretical and applied principles of that field.** These principles include the data structures used in knowledge representation, the algorithms needed to apply that knowledge, and the languages and programming techniques used in their implementation.

However, this definition suffers from the fact that intelligence itself is not very well defined or understood. Although most of us are certain that we know intelligent behavior when we see it, it is doubtful that anyone could come close to defining intelligence in a way that would be specific enough to help in the evaluation of a supposedly intelligent computer program, while still capturing the vitality and complexity of the human mind [Luger 2009].

[Def. 2] **Artificial intelligence (AI) is the field that studies the synthesis and analysis of computational agents that act intelligently.** Let us examine each part of this definition.

An agent is something that acts in an environment; it does something. Agents include worms, dogs, thermostats, airplanes, robots, humans, companies, and countries. We are interested in what an agent does; that is, how it acts. We judge an agent by its actions. **An agent acts intelligently when**

- **What it does is appropriate for its circumstances and its goals, considering the short term and long-term consequences of its actions**
- **It is flexible to changing environments and changing goals**
- **It learns from experience**
- **It makes appropriate choices given its perceptual and computational limitations**

A Computational agent is an agent whose decisions about its actions can be explained in terms of computation. That is, **the decision can be broken down into primitive operations that can be implemented in a physical device.** This computation can take many forms. In humans this computation is carried out in “wetware”; in computers it is carried out in “hardware.” Although there are some agents that are arguably not computational, such as the wind and rain eroding a landscape, it is an open question whether all intelligent agents are computational [Poole and Mackworth].

What can AI do today?

A concise answer is difficult because there are so many activities in so many subfields. Here I sample a few applications;

Robotic vehicles: A driverless robotic car named STANLEY sped through the rough terrain of the Mojave dessert at 22 mph, finishing the 132-mile course first to win the 2005 DARPA Grand Challenge. STANLEY is a Volkswagen Touareg outfitted with cameras, radar, and laser rangefinders to sense the environment and onboard software to command the steering, braking, and acceleration.

Speech recognition: A traveller calling United Airlines to book a flight can have the entire conversation guided by an automated speech recognition and dialog management system.

Autonomous planning and scheduling: A hundred million miles from Earth, NASA's Remote Agent program became the first on-board autonomous planning program to control the scheduling of operations for a spacecraft. REMOTE AGENT such as Mars Exploration Rove (Opportunity) nicknamed Oppy, generated plans from high-level goals specified from the ground and monitored the execution of those plans—detecting, diagnosing, and recovering from problems as they occurred.

Game playing: IBM's DEEP BLUE became the first computer program to defeat the world champion in a chess match when it bested Garry Kasparov by a score of 3.5 to 2.5 in an exhibition match. Kasparov said that he felt a “new kind of intelligence” across the board from

Spam fighting: Each day, learning algorithms classify over a billion messages as spam, saving the recipient from having to waste time deleting what, for many users, could comprise 80% or 90% of all messages, if not classified away by algorithms. Because the spammers are continually updating their tactics, it is difficult for a static programmed approach to keep up, and learning algorithms work best.

Logistics planning: During the Persian Gulf crisis of 1991, U.S. forces deployed a Dynamic Analysis and Replanning Tool, DART, to do automated logistics planning and scheduling for transportation. This involved up to 50,000 vehicles, cargo, and people at a time, and had to account for starting points, destinations, routes, and conflict resolution among all parameters.

The AI planning techniques generated in hours a plan that would have taken weeks with older methods. The Defense Advanced Research Project Agency (DARPA) stated that this single

application more than paid back DARPA's 30-year investment in AI

Robotics: The iRobot Corporation has sold over two million Roomba robotic vacuum cleaners for home use. The company also deploys the more rugged PackBot to Iraq and Afghanistan, where it is used to handle hazardous materials, clear explosives, and identify the location of snipers.

Machine Translation: A computer program automatically translates from Arabic to English, allowing an English speaker to see the headline “Ardogan Confirms That Turkey Would Not Accept Any Pressure, Urging Them to Recognize Cyprus.” The program uses a statistical model built from examples of Arabic-to-English translations and from examples of English text totalling two trillion words. None of the computer scientists on the team speak Arabic, but they do understand statistics and machine learning algorithms

These are just a few examples of artificial intelligence systems that exist today. Not magic or science fiction—but rather science, engineering, and mathematics. The two most fundamental concerns of AI researchers are knowledge representation and search, to which this course provides an introduction. The first of these addresses the problem of capturing in a language, i.e., one suitable for computer manipulation, the full range of knowledge required for intelligent behaviour. Search is a problem solving technique that systematically explores a space of problem states, i.e., successive and alternative stages in the problem-solving process

Artificial Neural Networks (ANN): is a computational model inspired by the human brain's structure, consisting of interconnected nodes (artificial neurons) organized in layers. ANNs process data by passing signals through these layers, using weights to learn patterns and make predictions or classifications. They are fundamental to machine learning and AI, enabling applications like image recognition and financial forecasting.

Why Neural Networks And Why now?

- As modern computers become ever more powerful, scientists continue to be challenged to use machines effectively for tasks that are relatively simple of humans.
- We learn easily to recognize the letter A or distinguish a cat from a bird.
- More experience allows us to refine our responses and improve our performance.
- Even without a teacher, we can group similar patterns together.

Brains versus Computers : Some numbers

1. There are approximately 10 billion neurons in the human cortex, compared with 10 of thousands of processors in the most powerful parallel computers.
2. Each biological neuron is connected to several thousands of other neurons, similar to the connectivity in powerful parallel computers.
3. Lack of processing units can be compensated by speed. The typical operating speeds of biological neurons is measured in milliseconds (10^{-3} s), while a silicon chip can operate in nanoseconds (10^{-9} s).
4. The human brain is extremely energy efficient, using approximately 10^{-16} joules per operation per second, whereas the best computers today use around 10^{-6} joules per operation per second.
5. Brains have been evolving for tens of millions of years, computers have been evolving for tens of decades.

What is a Neural Net. ?

An artificial neural network is an information-processing system that has certain performance characteristics in common with biological neural networks. Artificial neural networks have been developed as generalizations of mathematical models of human cognition or neural biology, based on the assumptions that:

1. Information processing occurs at many simple elements called neurons.
2. Signals are passed between neurons over connection links.
3. Each connection link has an associated weight, which, in a typical neural net, multiplies the signal transmitted.
4. Each neuron applies an activation function (usually nonlinear) to its net input (sum of weighted input signals) to determine its output signal.

A neural network is characterized by:

- (1) its pattern of connections between the neurons (called its **architecture**)
- (2) its method of determining the weights on the connections (called its **training, or learning, algorithm**)
- (3) its **activation function**.

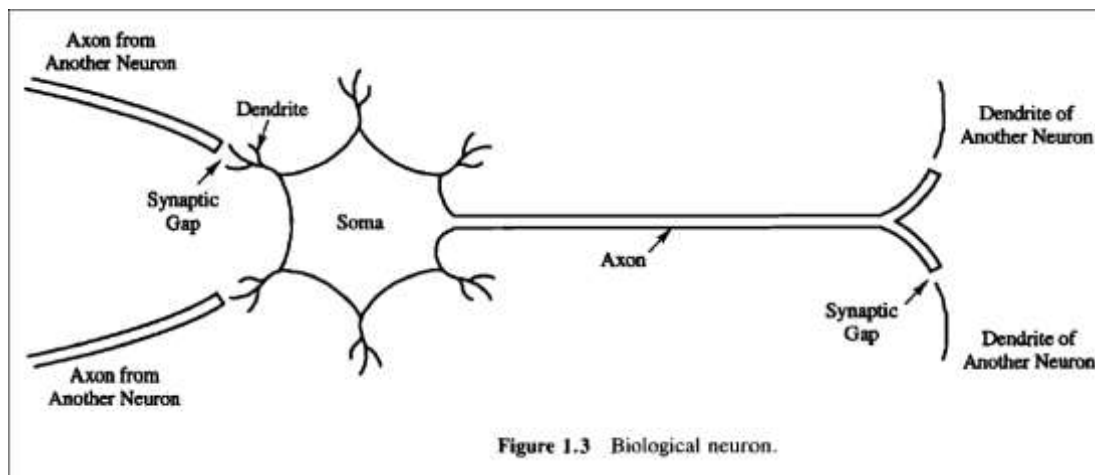
key features

- Several key features of the processing elements of artificial neural networks are:
- The processing element receives **many signals**.

- The processing element **sums the weighted** inputs.
- Under appropriate circumstances (sufficient input), the neuron **transmits a single output**.
- The output from a particular neuron may **go to many other neurons** (the axon branches).

Biological Neural Networks

- A biological neuron has three types of components that are of particular interest in understanding an artificial neuron: its **dendrites**, **soma**, and **axon**.
- The many dendrites receive signals from other neurons. The signals are electric impulses that are transmitted across a synaptic gap by means of a **chemical process**.
- The action of the chemical transmitter **modifies the incoming signal** (typically, by scaling the frequency of the signals that are received) in a manner similar to the action of the weights in an artificial neural network.



If-Then Rules

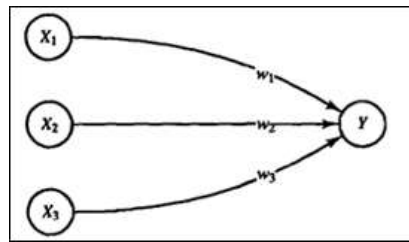
Where Are Neural Nets Being used?

1. Signal Processing.
2. Control.
3. Pattern Recognition.
4. Medicine.
5. Speech Production.
6. Speech Recognition.
7. Businesses

Activation or Activity Level

For example, consider a neuron Y , that receives inputs from neurons X_1 , X_2 , and X_3 . The weights on the connections from X_1 , X_2 , and X_3 to neuron Y are w_1 to w_2 , and w_3 , respectively. The net input, y_{in} , to neuron Y is the sum of the weighted signals from neurons X_1 , X_2 , and X_3 , i.e.

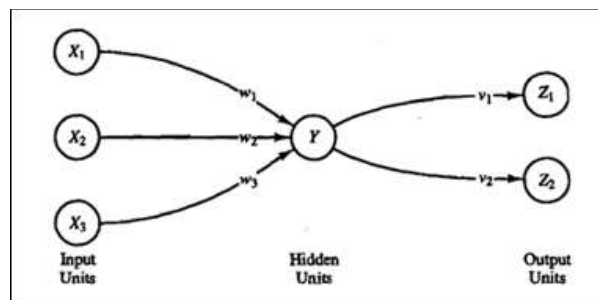
$$y_{in} = W_1X_1 + W_2X_2 + W_3X_3$$



$$f(x) = \frac{1}{1 + \exp(-x)}$$

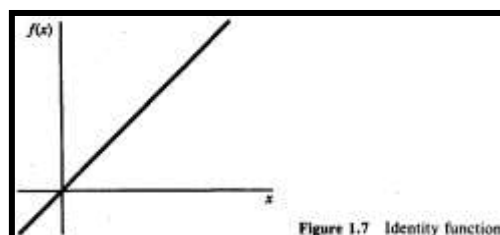
The activation y of neuron Y is given by some function of its net input, $y = f(y_{in})$, e.g., the logistic sigmoid function (an S-shaped curve)

Now suppose further that neuron Y is connected to neurons Z_1 and Z_2 , with weights v_1 and v_2 , respectively. Neuron Y sends its signal y to each of these units. However, in general, the values received by neurons Z_1 and Z_2 will be different, because each signal is scaled by the appropriate weight, v_1 or v_2 . In a typical net, the activations Z_1 and Z_2 of neurons Z_1 and Z_2 would depend on inputs from several or even many neurons, not just one, as shown in this simple example.



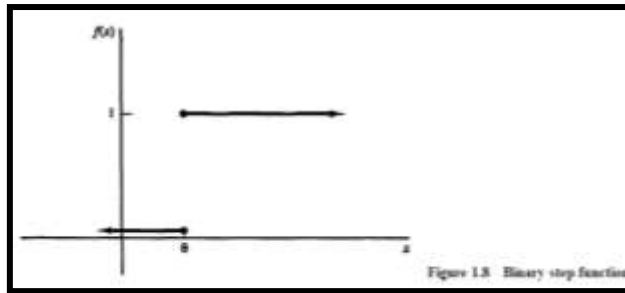
Common Activation Functions

1. Identity function: $f(x) = x$ for all x



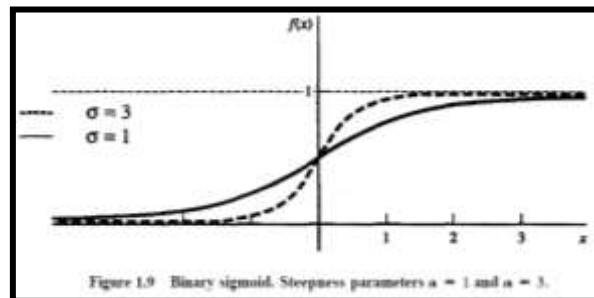
$f(x) = x$ for all x

2. Binary step function (with threshold Θ):



$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases}$$

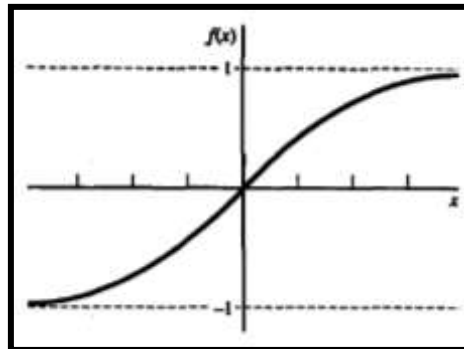
3. Binary sigmoid:



$$f(x) = \frac{1}{1 + \exp(-\sigma x)}$$

$$f'(x) = \sigma f(x)[1 - f(x)].$$

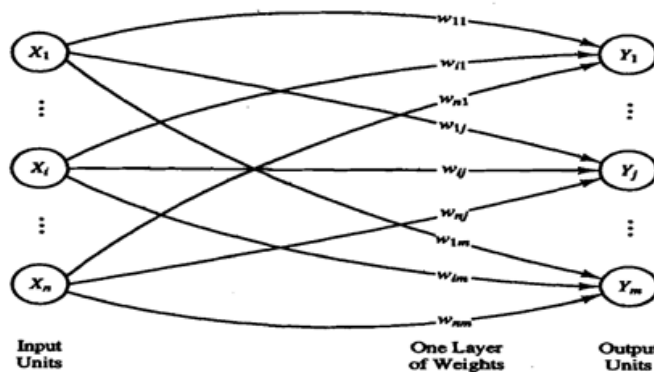
4. Bipolar sigmoid:



$$\begin{aligned} g(x) &= 2f(x) - 1 = \frac{2}{1 + \exp(-\sigma x)} - 1 \\ &= \frac{1 - \exp(-\sigma x)}{1 + \exp(-\sigma x)}. \\ g'(x) &= \frac{\sigma}{2} [1 + g(x)][1 - g(x)]. \end{aligned}$$

Typical Architectures

- The arrangement of neurons into layers and the connection patterns within and between layers is called the **net architecture**.
- **Feed forward nets**: nets in which the signals flow from the input units to the output units, in a forward direction.
- **Recurrent net**: in which there are closed-loop signal paths from a unit back to itself.
- **A single-layer net**: has one layer of connection weights. Often, the units can be distinguished as input units, which receive signals from the outside world, and output units, from which the response of the net can be read.



Setting the Weights

- In addition to the architecture, the method of setting the values of the weights (**training**) is an important distinguishing characteristic of different neural nets.
- The weights are then adjusted according to a learning algorithm. This process is known as **supervised training**.
- **Unsupervised training** Self-organizing neural nets group similar input vectors together without the use of training data to specify what a typical member of each group looks like or to which group each vector belongs.

Lec. 2

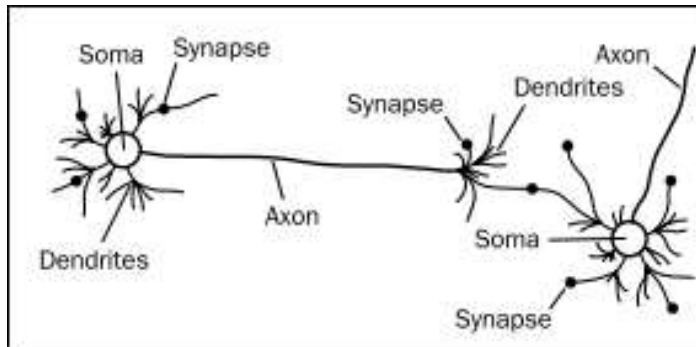


Table 6.1 Analogy between biological and artificial neural networks

Biological neural network	Artificial neural network
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

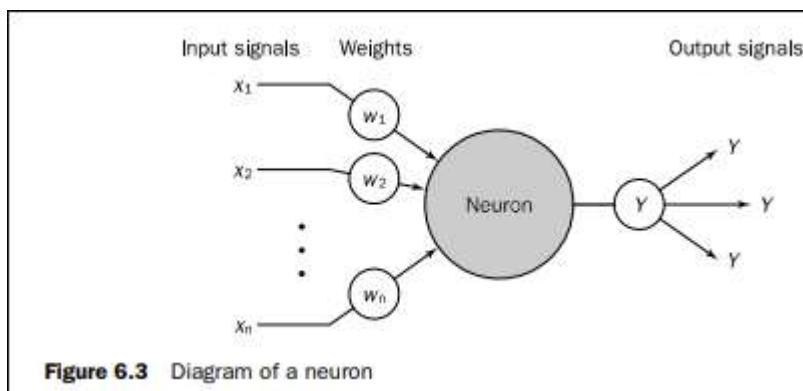


Figure 6.3 Diagram of a neuron

$$X = \sum_{i=1}^n x_i w_i$$

$$Y = \begin{cases} +1 & \text{if } X \geq \theta \\ -1 & \text{if } X < \theta \end{cases}$$

$$Y = \text{sign} \left[\sum_{i=1}^n x_i w_i - \theta \right]$$

McCulloch-Pitts Neuron Model

The first formal definition of a synthetic neuron model based on the highly simplified considerations of the biological model described in the preceding section was formulated by McCulloch and Pitts (1943). The McCulloch-Pitts model of the neuron is shown in Figure 2.3a. The inputs x_i , for $i = 1, 2, \dots, n$, are 0 or 1, depending on the absence or presence of the input impulse at instant k . The neuron's output signal is denoted as o . The firing rule for this model is defined as follows

$$o^{k+1} = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i^k \geq T \\ 0 & \text{if } \sum_{i=1}^n w_i x_i^k < T \end{cases}$$

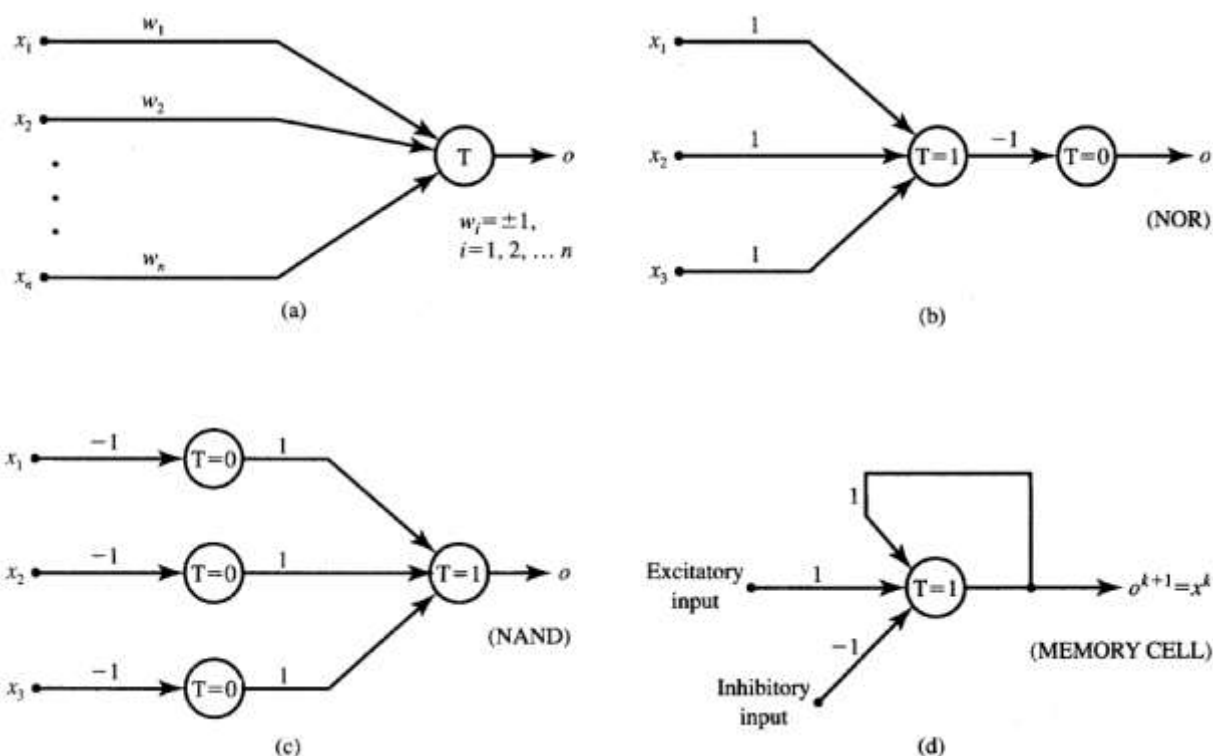


Figure 2.3 McCulloch-Pitts model neuron and elementary logic networks: (a) model diagram, (b) NOR gate, (c) NAND gate, and (d) memory cell.

Also, the steps in this method are very similar to how Neural Networks learn, which is as follows;

- Initialize weight values and bias
- Forward Propagate
- Check the error
- Backpropagate and Adjust weights and bias
- Repeat for all training examples

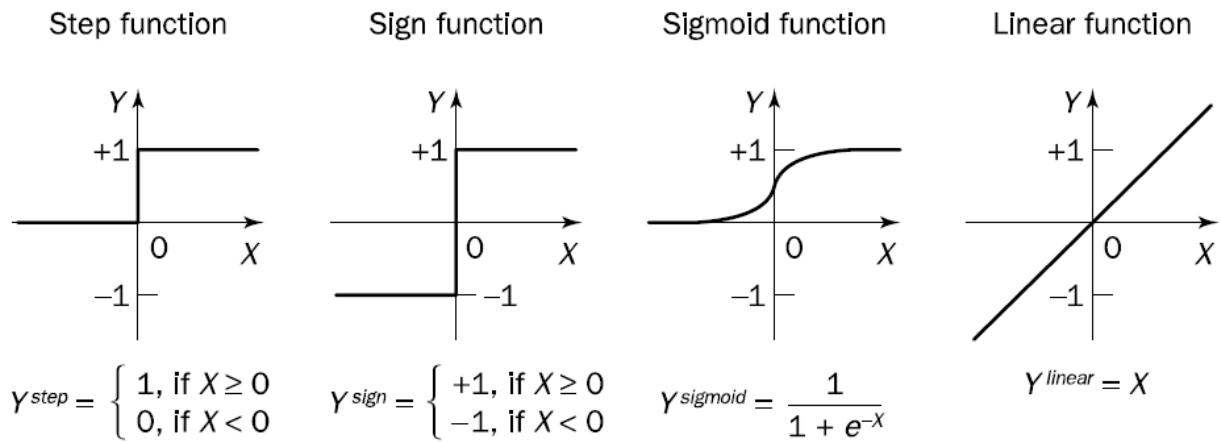


Figure 6.4 Activation functions of a neuron

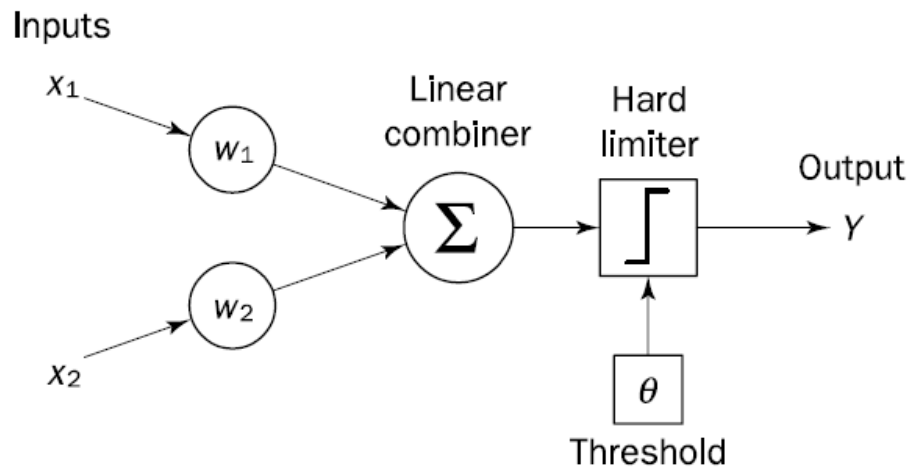
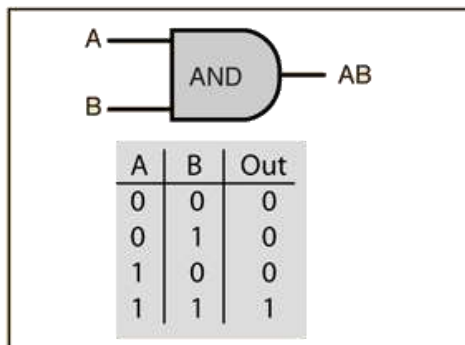


Figure 6.5 Single-layer two-input perceptron

Neural Representation of AND, OR, NOT, XOR and XNOR Logic Gates (Perceptron Algorithm)

AND Gate

From our knowledge of logic gates, we know that an AND logic table is given by the diagram below



The question is, what are the weights and bias for the AND perceptron?

First, we need to understand that the output of an AND gate is 1 only if both inputs (in this case, x_1 and x_2) are 1. So, following the steps listed above;

Row 1

- From $w_1 \cdot x_1 + w_2 \cdot x_2 + b$, initializing w_1 , w_2 , as 1 and b as -1 , we get;
 $x_1(1) + x_2(1) - 1$

- Passing the first row of the AND logic table ($x_1=0$, $x_2=0$), we get;
 $0+0-1 = -1$

- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. Therefore, this row is correct, and no need for Backpropagation.

Row 2

- Passing ($x_1=0$ and $x_2=1$), we get;
 $0+1-1 = 0$

- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. This row is correct, as the output is 0 for the AND gate.
- From the Perceptron rule, this works (for both row 1, row 2 and 3).

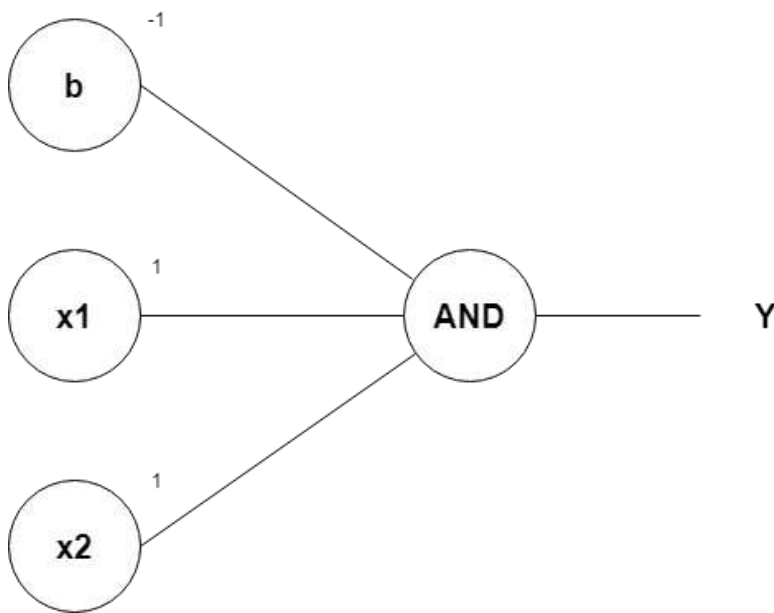
Row 4

- Passing ($x_1=1$ and $x_2=1$), we get;
 $1+1-1 = 1$

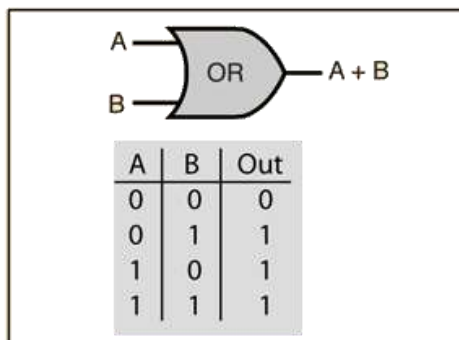
- Again, from the perceptron rule, this is still valid.

Therefore, we can conclude that the model to achieve an AND gate, using the Perceptron algorithm is;

$$x_1 + x_2 - 1$$



OR Gate



From the diagram, the OR gate is 0 only if both inputs are 0.

Row 1

- From $w_1x_1 + w_2x_2 + b$, initializing w_1, w_2 , as 1 and b as -1 , we get;

$$x_1(1) + x_2(1) - 1$$

- Passing the first row of the OR logic table ($x_1=0, x_2=0$), we get;

$$0 + 0 - 1 = -1$$

- From the Perceptron rule, if $Wx + b \leq 0$, then $y' = 0$. Therefore, this row is correct.

Row 2

- Passing ($x_1=0$ and $x_2=1$), we get;

$$0 + 1 - 1 = 0$$

- From the Perceptron rule, if $Wx + b \leq 0$, then $y' = 0$. Therefore, this row is incorrect.
- So we want values that will make inputs $x_1=0$ and $x_2=1$ give y' a value of 1. If we change w_2 to 2, we have;

$$0 + 2 - 1 = 1$$

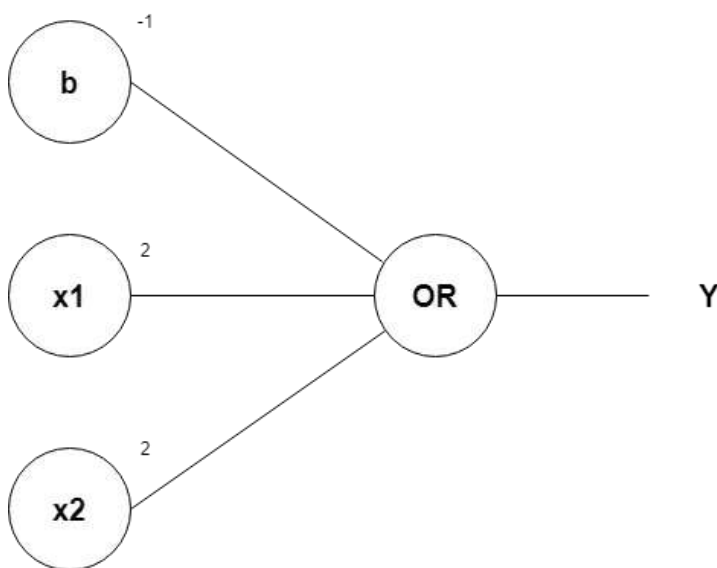
- From the Perceptron rule, this is correct for both the row 1 and 2.

Row 3

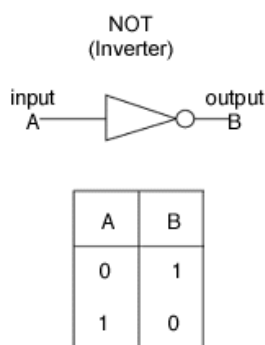
- Passing ($x_1=1$ and $x_2=0$), we get;
 $1+0-1 = 0$
- From the Perceptron rule, if $Wx+b \leq 0$, then $y^*=0$. Therefore, this row is incorrect.
- Since it is similar to that of row 2, we can just change w_1 to 2, we have;
 $2+0-1 = 1$
- From the Perceptron rule, this is correct for both the row 1, 2 and 3.

Row 4

- Passing ($x_1=1$ and $x_2=1$), we get;
 $2+2-1 = 3$
 - Again, from the perceptron rule, this is still valid. Quite Easy!
- Therefore, we can conclude that the model to achieve an OR gate, using the Perceptron algorithm is;
 $2x_1+2x_2-1$



NOT Gate



From the diagram, the output of a NOT gate is the inverse of a single input. So, following the steps listed above;

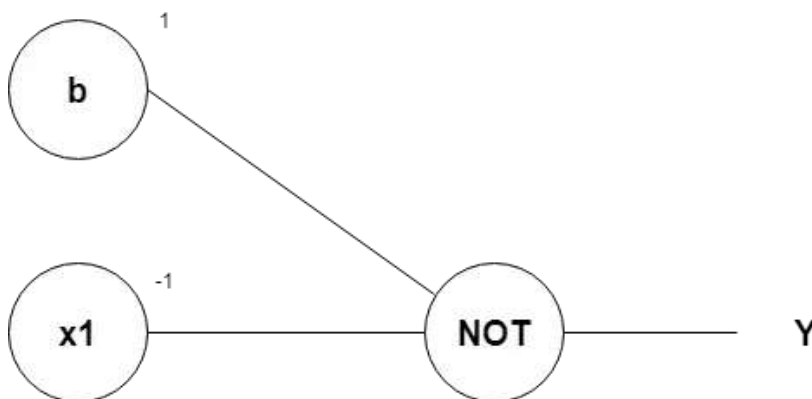
Row 1

- From w_1x_1+b , initializing w_1 as 1 (since single input), and b as -1 , we get;
 $x_1(1)-1$
- Passing the first row of the NOT logic table ($x_1=0$), we get;
 $0-1 = -1$
- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. This row is incorrect, as the output is 1 for the NOT gate.
- So we want values that will make input $x_1=0$ to give y' a value of 1. If we change b to 1, we have;
 $0+1 = 1$
- From the Perceptron rule, this works.

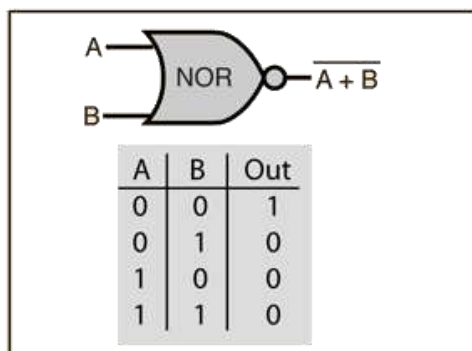
Row 2

- Passing ($x_1=1$), we get;
 $1+1 = 2$
- From the Perceptron rule, if $Wx+b > 0$, then $y'=1$. This row is so incorrect, as the output is 0 for the NOT gate.
- So we want values that will make input $x_1=1$ to give y' a value of 0. If we change w_1 to -1 , we have;
 $-1+1 = 0$
- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. Therefore, this works (for both row 1 and row 2).

Therefore, we can conclude that the model to achieve a NOT gate, using the Perceptron algorithm is;
 $-x_1+1$



NOR Gate



From the diagram, the NOR gate is 1 only if both inputs are 0.

Row 1

- From $w_1x_1 + w_2x_2 + b$, initializing w_1 and w_2 as 1, and b as -1, we get;

$$x_1(1) + x_2(1) - 1$$

- Passing the first row of the NOR logic table ($x_1=0, x_2=0$), we get;

$$0 + 0 - 1 = -1$$

- From the Perceptron rule, if $Wx + b \leq 0$, then $y^* = 0$. This row is incorrect, as the output is 1 for the NOR gate.
- So we want values that will make input $x_1=0$ and $x_2 = 0$ to give y^* a value of 1. If we change b to 1, we have;

$$0 + 0 + 1 = 1$$

- From the Perceptron rule, this works.

Row 2

- Passing ($x_1=0, x_2=1$), we get;

$$0 + 1 + 1 = 2$$

- From the Perceptron rule, if $Wx + b > 0$, then $y^* = 1$. This row is incorrect, as the output is 0 for the NOR gate.
- So we want values that will make input $x_1=0$ and $x_2 = 1$ to give y^* a value of 0. If we change w_2 to -1, we have;

$$0 - 1 + 1 = 0$$

- From the Perceptron rule, this is valid for both row 1 and row 2.

Row 3

- Passing ($x_1=1, x_2=0$), we get;

$$1 + 0 + 1 = 2$$

- From the Perceptron rule, if $Wx + b > 0$, then $y^* = 1$. This row is incorrect, as the output is 0 for the NOR gate.
- So we want values that will make input $x_1=0$ and $x_2 = 1$ to give y^* a value of 0. If we change w_1 to -1, we have;

$$-1 + 0 + 1 = 0$$

- From the Perceptron rule, this is valid for both row 1, 2 and 3.

Row 4

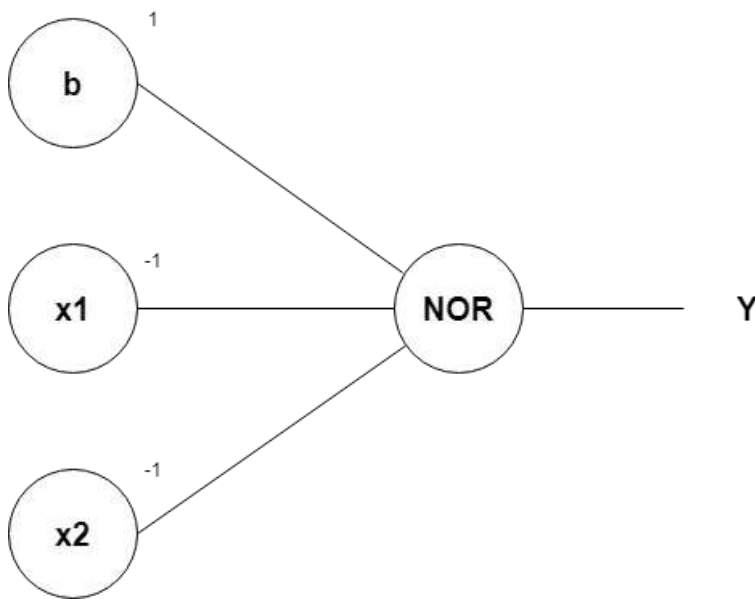
- Passing ($x_1=1, x_2=1$), we get;

$$-1 - 1 + 1 = -1$$

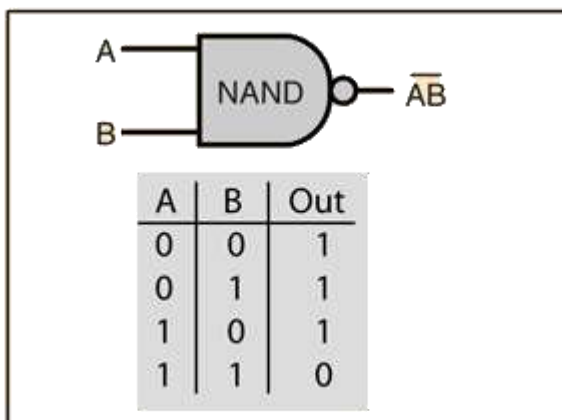
- From the Perceptron rule, this still works.

Therefore, we can conclude that the model to achieve a NOR gate, using the Perceptron algorithm is;

$$-x_1 - x_2 + 1$$



NAND Gate



From the diagram, the NAND gate is 0 only if both inputs are 1.

Row 1

- From $w_1x_1 + w_2x_2 + b$, initializing w_1 and w_2 as 1, and b as -1, we get;
 $x_1(1) + x_2(1) - 1$

- Passing the first row of the NAND logic table ($x_1=0, x_2=0$), we get;
 $0+0-1 = -1$

- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. This row is incorrect, as the output is 1 for the NAND gate.
- So we want values that will make input $x_1=0$ and $x_2=0$ to give y' a value of 1. If we change b to 1, we have;

$$0+0+1 = 1$$

- From the Perceptron rule, this works.

Row 2

- Passing ($x_1=0, x_2=1$), we get;

$$0+1+1 = 2$$

- From the Perceptron rule, if $Wx+b > 0$, then $y'=1$. This row is also correct (for both row 2 and row 3).

Row 4

- Passing $(x1=1, x2=1)$, we get;

$$1+1+1 = 3$$

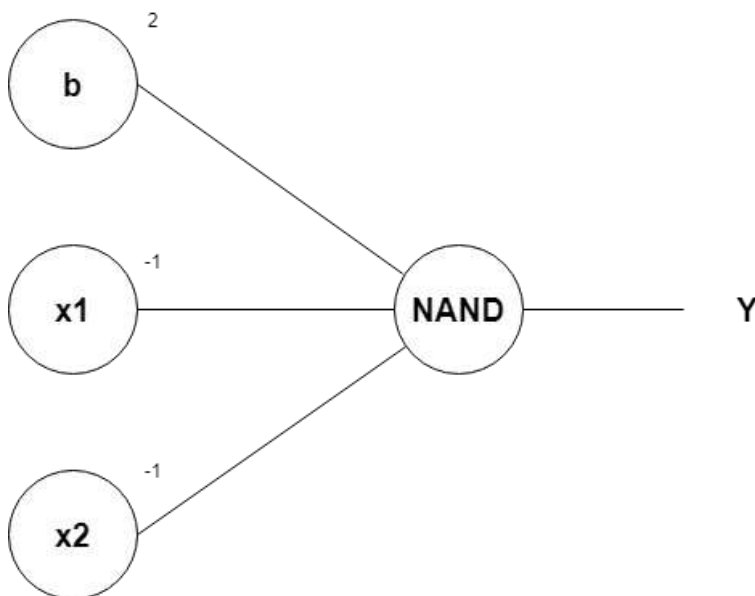
- This is not the expected output, as the output is 0 for a NAND combination of $x1=1$ and $x2=1$.
- Changing values of $w1$ and $w2$ to -1 , and value of b to 2 , we get;

$$-1-1+2 = 0$$

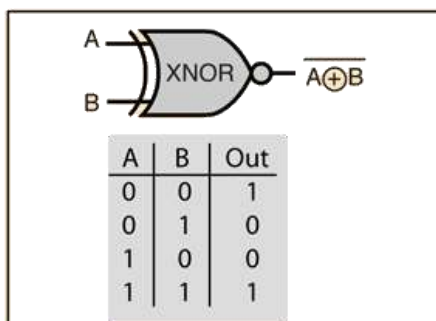
- It works for all rows.

Therefore, we can conclude that the model to achieve a NAND gate, using the Perceptron algorithm is;

$$-x1-x2+2$$



XNOR Gate



Now that we are done with the necessary basic logic gates, we can combine them to give an XNOR gate.

The boolean representation of an XNOR gate is;

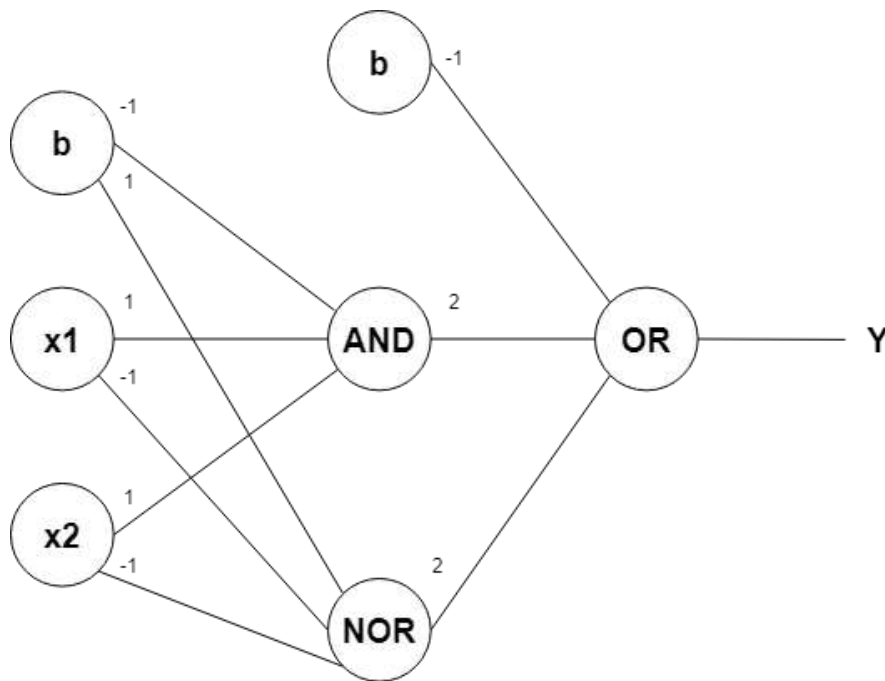
$$x_1x_2 + x_1x_2'$$

Where ' means inverse.

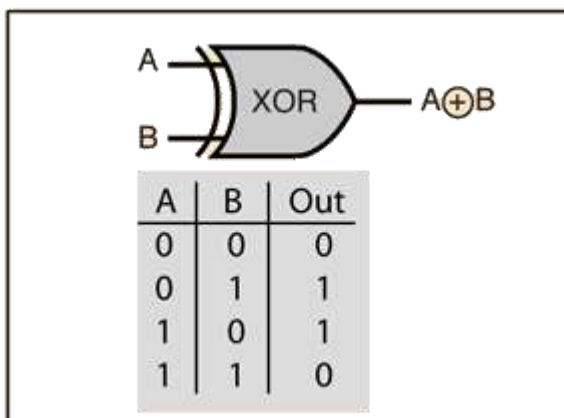
From the expression, we can say that the XNOR gate consists of an AND gate (x_1x_2), a NOR gate (x_1x_2'), and an OR gate.

This means we will have to combine 3 perceptrons:

- AND (x_1+x_2-1)
- NOR ($-x_1-x_2+1$)
- OR ($2x_1+2x_2-1$)



XOR Gate



The boolean representation of an XOR gate is;

$$x_1x_2' + x_1'x_2$$

We first simplify the boolean expression

$$x_1'x_2 + x_1x_2' + x_1'x_1 + x_2x_2$$

$$x_1(x_1' + x_2') + x_2(x_1' + x_2)$$

$$(x_1 + x_2)(x_1' + x_2')$$

$$(x_1 + x_2)(x_1x_2)'$$

From the simplified expression, we can say that the XOR gate consists of an OR gate ($x_1 + x_2$), a NAND gate ($-x_1-x_2+1$) and an AND gate ($x_1+x_2-1.5$).

This means we will have to combine 2 perceptrons:

- OR ($2x_1+2x_2-1$)
- NAND ($-x_1-x_2+2$)
- AND (x_1+x_2-1)

The boolean representation of an XOR gate is;

$$x_1x_2' + x_1'x_2$$

We first simplify the boolean expression

$$x_1'x_2 + x_1x_2' + x_1'x_1 + x_2x_2$$

$$x_1(x_1' + x_2') + x_2(x_1' + x_2)$$

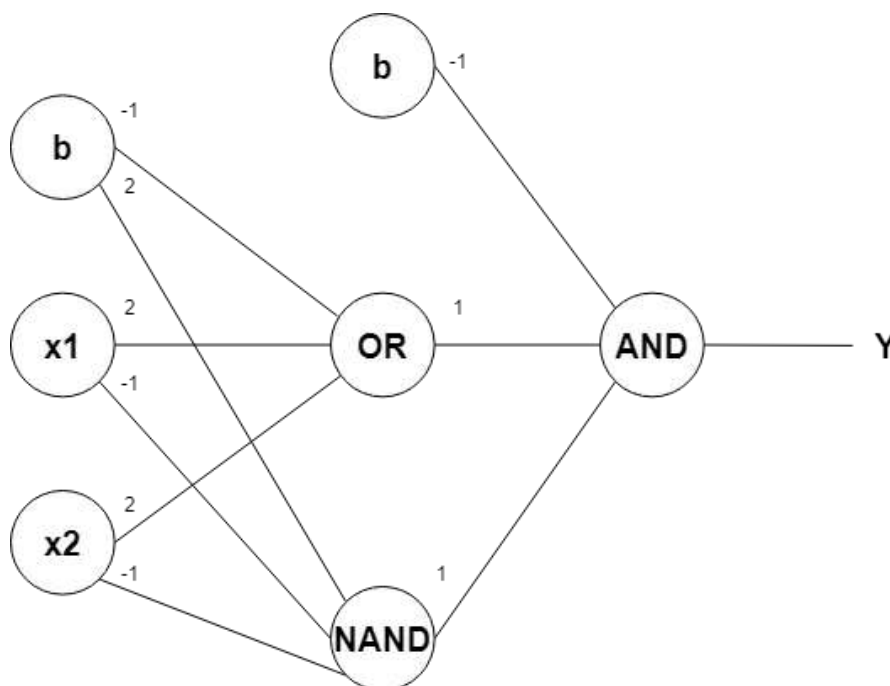
$$(x_1 + x_2)(x_1' + x_2')$$

$$(x_1 + x_2)(x_1x_2)'$$

From the simplified expression, we can say that the XOR gate consists of an OR gate ($x_1 + x_2$), a NAND gate ($-x_1-x_2+1$) and an AND gate ($x_1+x_2-1.5$).

This means we will have to combine 2 perceptrons:

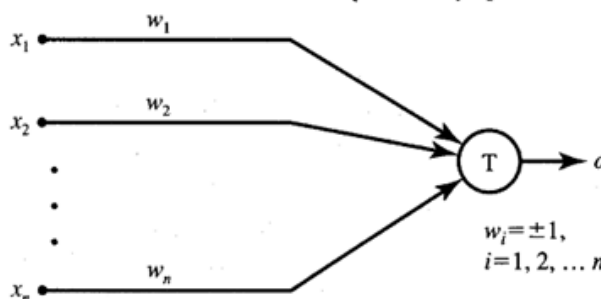
- OR ($2x_1+2x_2-1$)
- NAND ($-x_1-x_2+2$)
- AND (x_1+x_2-1)



THE McCULLOCH-PITTS NEURON

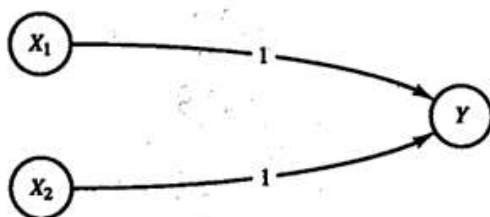
- The first formal definition of a synthetic neuron model based on the highly simplified considerations of the biological model described in the preceding section was formulated by McCulloch and Pitts (1943). The inputs x_i , for $i = 1, 2, \dots, n$, are 0 or 1, depending on the absence or presence of the input impulse at instant k . The neuron's output signal is denoted as o . The firing rule for this model is defined as follows

$$o^{k+1} = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i^k \geq T \\ 0 & \text{if } \sum_{i=1}^n w_i x_i^k < T \end{cases}$$



Logic Functions

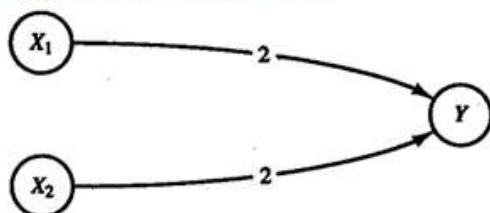
- AND
- The AND function gives the response "true" if both input values are "true"; otherwise the response is "false." If we represent "true" by the value 1, and "false" by 0, this gives the following four training input, target output pairs:



x_1	x_2	$\rightarrow$	y
1	1		1
1	0		0
0	1		0
0	0		0

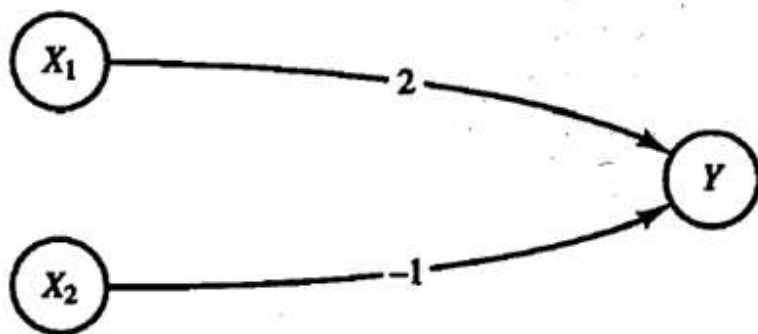
OR gate

The threshold on unit Y is 2.



x_1	x_2	$\rightarrow$	y
1	1		1
1	0		1
0	1		1
0	0		0

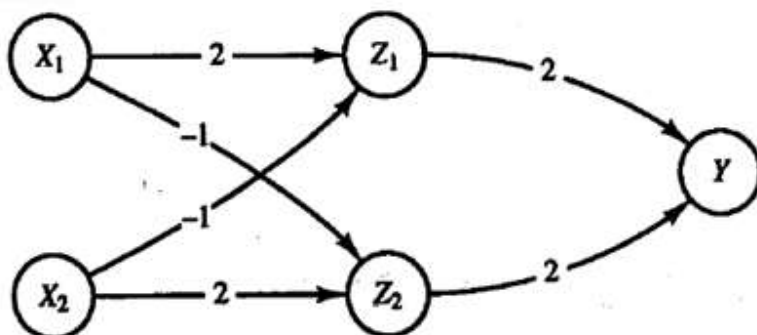
AND NOT



x_1	x_2	$\rightarrow y$
1	1	0
1	0	1
0	1	0
0	0	0

XOR

$$x_1 \text{ XOR } x_2 \leftrightarrow (x_1 \text{ AND NOT } x_2) \text{ OR } (x_2 \text{ AND NOT } x_1).$$



x_1	x_2	$\rightarrow y$
1	1	0
1	0	1
0	1	1
0	0	0

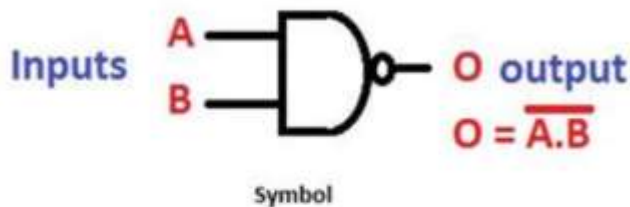
$$z_1 = x_1 \text{ AND NOT } x_2$$

$$z_2 = x_2 \text{ AND NOT } x_1.$$

$$y = z_1 \text{ OR } z_2.$$

Lec. 4

NAND Gate



Inputs		Output
A	B	O
0	0	1
0	1	1
1	0	1
1	1	0

Truth table

From the diagram, the NAND gate is only 0 if both inputs are ones.

From $(w_1 \cdot A + w_2 \cdot B) + b$, initializing w_1, w_2 as 1 and b as -1, we get $(1)A + (1)B - 1$.

ROW 1:

Passing the first row of the AND logic table ($A=0, B=0$), we get $0 + 0 - 1 = -1$. From the Perceptron rule, if $wx + b \leq 0$, then $y = 0$. This row is incorrect, as the output is 1 for the NAND gate. So, we want value that will make inputs $A=0, B=0$ give y a value of 1. If we change b to 1, we have $0 + 0 + 1 = 1$. From the Perceptron rule, this works.

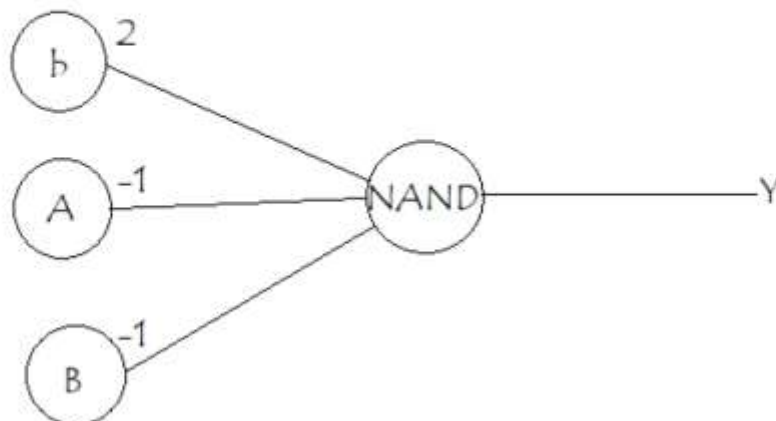
ROW 2:

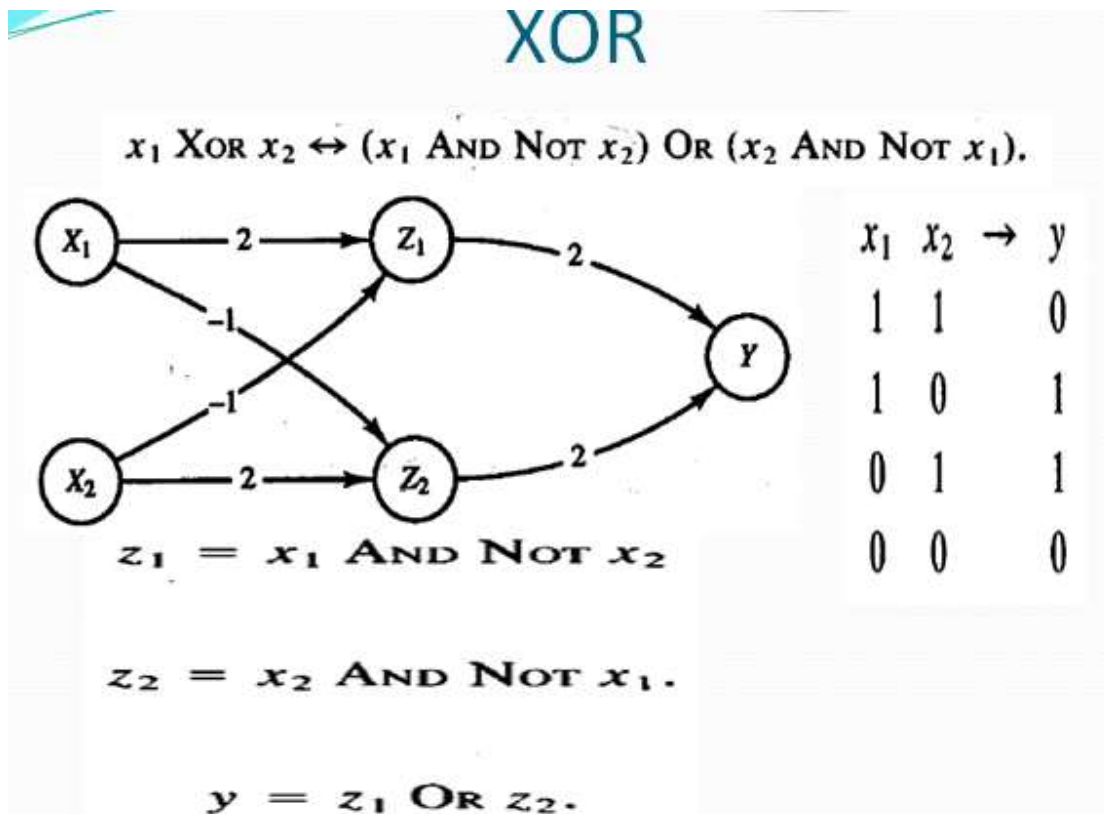
Passing ($A=0, B=1$) we get $0 + 1 + 1 = 2$. From the Perceptron rule, if $wx + b > 0$ then $y = 1$. This row is also correct for both row 2 and row 3.

ROW 4:

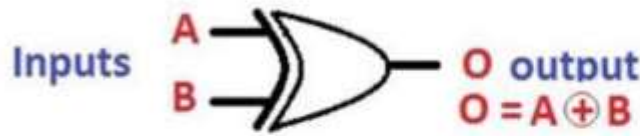
Passing ($A=1, B=1$) we get $1 + 1 + 1 = 3$. This is not the expected output, as the output is 0 for a NAND combination of $A=1$ and $B=1$. Changing the values of w_1 and w_2 to -1, and value of b to 2, we get $-1 - 1 + 2 = 0$. It works for all rows.

Therefore, we can conclude that the model to achieve a NAND gate, using Perceptron algorithm is: $-A - B + 2$.





XOR Gate



Symbol

Inputs		Output
A	B	O
0	0	0
0	1	1
1	0	1
1	1	0

Truth table

The Boolean representation of an XOR gate is

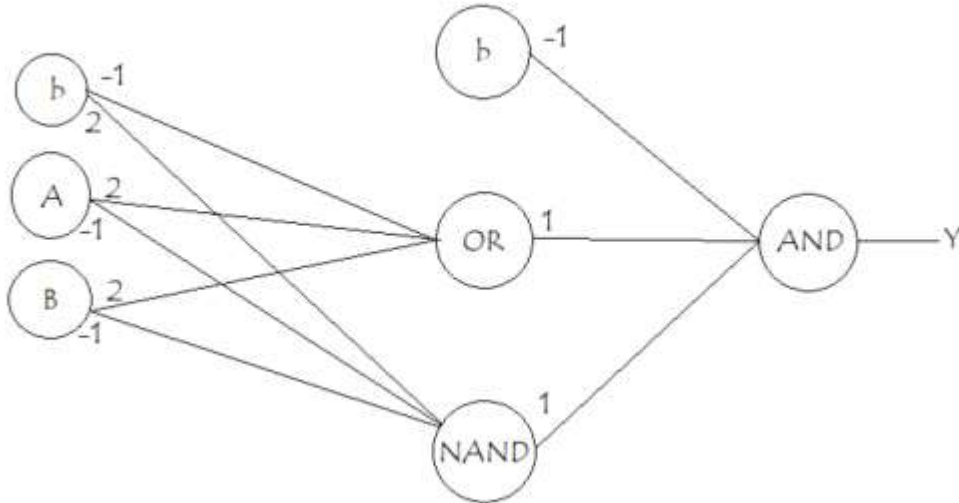
$$AB + \bar{A}B$$

We first simplify the Boolean expression

$$\begin{aligned} AB + \bar{A}B + A\bar{A} + \bar{B}B \\ AB + (\bar{A}B + A\bar{A}) + \bar{B}B \\ A(\bar{A} + \bar{B}) + B(\bar{A} + \bar{B}) \\ (A + B)(\bar{A} + \bar{B}) \\ (A + B)(\overline{AB}) \end{aligned}$$

From simplified expression, we can say that the XOR gate consists of an OR gate, a NAND gate, and an AND gate:

- AND ($A + B - 1$)
- OR ($2A + 2B - 1$)
- NAND ($-A - B + 2$)



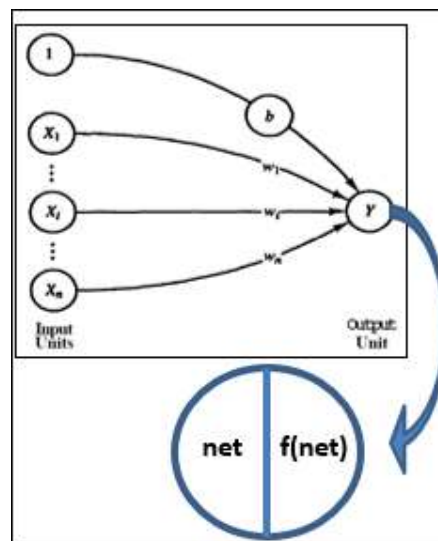
Hebb Learning

- 1- Architecture
- 2- Algorithm
- 3- Application / logic gate

Hebb learning

Donald Hebb, a psychologist at McGill University, designed the first learning law for artificial neural networks [Hebb, 1949]. His premise was that if two neurons were active simultaneously, then the strength of the connection between them should be increased. Refinements were subsequently made to this rather general statement to allow computer simulations [Rochester, Holland, Haibt & Duda, 1956]. The idea is closely related to the correlation matrix learning developed by Kohonen (1972) and Anderson (1972) among others. An expanded form of Hebb learning [McClelland & Rumelhart, 1988] in which units that are simultaneously off also reinforce the weight on the connection between them will be presented in Chapters 2 and 3.

Architecture



$$\text{net} = b + \sum_i x_i w_i.$$

$$f(\text{net}) = \begin{cases} 1 & \text{if } \text{net} \geq 0; \\ -1 & \text{if } \text{net} < 0; \end{cases}$$

2.2.1 Algorithm

Step 0. Initialize all weights:

$$w_i = 0 \quad (i = 1 \text{ to } n).$$

Step 1. For each input training vector and target output pair, $s : t$, do steps 2–4.

Step 2. Set activations for input units:

$$x_i = s_i \quad (i = 1 \text{ to } n).$$

Step 3. Set activation for output unit:

$$y = t.$$

Step 4. Adjust the weights for

$$w_i(\text{new}) = w_i(\text{old}) + x_i y \quad (i = 1 \text{ to } n).$$

Adjust the bias:

$$b(\text{new}) = b(\text{old}) + y.$$

Note that the bias is adjusted exactly like a weight from a "unit" whose output signal is always 1. The weight update can also be expressed in vector **form** as

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) + \mathbf{x}y.$$

This is often written in terms of the weight change, $\Delta \mathbf{w}$, as

$$\Delta \mathbf{w} = \mathbf{x}y$$

and

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) + \Delta \mathbf{w}.$$

There are several methods of implementing the Hebb rule for learning. The foregoing algorithm requires only one pass through the training set; other equivalent methods of finding the weights are described in Section 3.1.1, where the Hebb rule for pattern association (in which the target is a vector) is presented.

2.2.2 Application

Bias types of inputs are not explicitly used in the original formulation of Hebb learning. However, they are included in the examples in this section (shown as a third input component that is always 1) because without them, the problems discussed cannot be solved.

Logic functions

Example 2.5 A Hebb net for the AND function: binary inputs and targets

INPUT			TARGET
x_1	x_2	1	
1	1	1	1
1	0	1	0
0	1	1	0
0	0	1	0

For each training input: target, the weight change is the product of the input vector and the target value, i.e.,

- 1- Identify how many training sample we should consider ?

What is mean training sample ? answer how many inputs are there ? x_1, x_2, x_3, x_4 4 training samples

Second question .. why we are considered training samples ?

Answer because on this training samples the iterations means how many times this algorithm will iterate it depends. so, 4 training samples means 4 iterations

- 2- Activation function (bipolar sigmoid $(-1$ to $1)$) if inputs are given bipolar $(-1$ to $1)$

Hebbian (how weights adjusted)

Set all weight and bias $=0$

For each training input: target, the weight change is the product of the input vector and the target value, i.e.,

$$\Delta w_1 = x_1 t, \quad \Delta w_2 = x_2 t, \quad \Delta b = t.$$

The new weights are the sum of the previous weights and the weight change. Only one iteration through the training vectors is required. The weight updates for the first input are as follows:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(0 \ 0 \ 0)$
$(1 \ 1 \ 1)$	1	$(1 \ 1 \ 1)$	$(1 \ 1 \ 1)$

The separating line (see Section 2.1.3) becomes

$$x_2 = -x_1 - 1.$$

The graph, presented in Figure 2.7, shows that the response of the net will now be correct for the first input pattern. Presenting the second, third, and fourth training inputs shows that because the target value is 0, no learning occurs. Thus, using binary target values prevents the net from learning any pattern for which the target is "off":

The separating line (see Section 2.1.3) becomes

$$x_2 = -x_1 - 1.$$

The graph, presented in Figure 2.7, shows that the response of the net will now be correct for the first input pattern. Presenting the second, third, and fourth training inputs shows that because the target value is 0, no learning occurs. Thus, using binary target values prevents the net from learning any pattern for which the target is "off":

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
$(1 \ 0 \ 1)$	0	$(0 \ 0 \ 0)$	$(1 \ 1 \ 1)$
$(0 \ 1 \ 1)$	0	$(0 \ 0 \ 0)$	$(1 \ 1 \ 1)$
$(0 \ 0 \ 1)$	0	$(0 \ 0 \ 0)$	$(1 \ 1 \ 1)$

Example 2.6 A Hebb net for the AND function: binary inputs bipolar targets

INPUT	TARGET
$(x_1 \ x_2 \ 1)$	
$(1 \ 1 \ 1)$	1
$(1 \ 0 \ 1)$	-1
$(0 \ 1 \ 1)$	-1
$(0 \ 0 \ 1)$	-1

Presenting the first input, including a value of 1 for the third component, yields the following:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(0 \ 0 \ 0)$
$(1 \ 1 \ 1)$	1	$(1 \ 1 \ 1)$	$(1 \ 1 \ 1)$

The separating line becomes

$$x_2 = -x_1 - 1.$$

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
$(1 \ 0 \ 1)$	-1	$(-1 \ 0 \ -1)$	$(0 \ 1 \ 0)$
$(0 \ 1 \ 1)$	-1	$(0 \ -1 \ -1)$	$(0 \ 0 \ -1)$
$(0 \ 0 \ 1)$	-1	$(0 \ 0 \ -1)$	$(0 \ 0 \ -2)$

However, these weights do not provide the correct response for the first input pattern.

Example 2.7 A Hebb net for the AND function: bipolar inputs and targets

INPUT	TARGET
$(x_1 \ x_2 \ 1)$	
$(1 \ 1 \ 1)$	1
$(1 \ -1 \ 1)$	-1
$(-1 \ 1 \ 1)$	-1
$(-1 \ -1 \ 1)$	-1

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(0 \ 0 \ 0)$
$(1 \ 1 \ 1)$	1	$(1 \ 1 \ 1)$	$(1 \ 1 \ 1)$

The separating line becomes

$$x_2 = -x_1 - 1.$$

The graph in Figure 2.9 shows that the response of the net will now be correct for the first input point (and also, by the way, for the input point $(-1, -1)$). Presenting the second input vector and target results in the following situation:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(1 \ 1 \ 1)$
$(1 \ -1 \ 1)$	-1	$(-1 \ 1 \ -1)$	$(0 \ 2 \ 0)$

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(0 \ 2 \ 0)$
$(-1 \ 1 \ 1)$	-1	$(1 \ -1 \ -1)$	$(1 \ 1 \ -1)$

The separating line becomes

$$x_2 = -x_1 + 1.$$

The graph in Figure 2.11 shows that the response of the net will now be correct for the first three input points (and also, by the way, for the input point $(-1, -1)$). Presenting the last point, we obtain the following:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(1 \ 1 \ -1)$
$(-1 \ -1 \ 1)$	-1	$(1 \ 1 \ -1)$	$(2 \ 2 \ -2)$

Even though the weights have changed, the separating line is still

$$x_2 = -x_1 + 1,$$

so the graph of the decision regions (the positive response and the negative response) remains as in Figure 2.11.

Binary i/p binary target is not correct to get AND gate as a target

Binary i/p bipolar target is not correct to get AND gate as a target

Bipolar i/p Bipolar target is correct to get AND gate as a target

Homework:

- 1- Hebb net , OR function , bipolar i/p binary target
- 2- Hebb net , OR function , Bipolar i/p , and target

Lec.5

Unsupervised learning: A means of modifying the weights of a neural net without specifying the desired output for any input patterns. Used in self-organizing neural nets for clustering data, extracting principal components, or curve fitting.

Supervised training: Process of adjusting the weights in a neural net using a learning algorithm; the desired output for each of a set of training input vectors is presented to the net. Many iterations through the training data may be required.

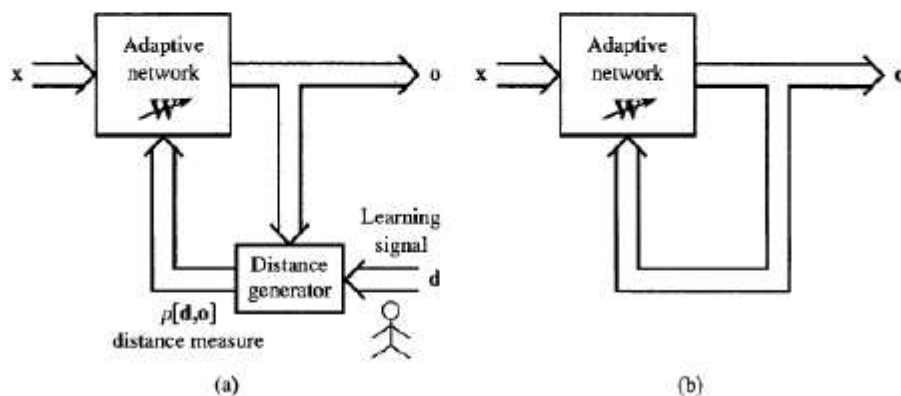


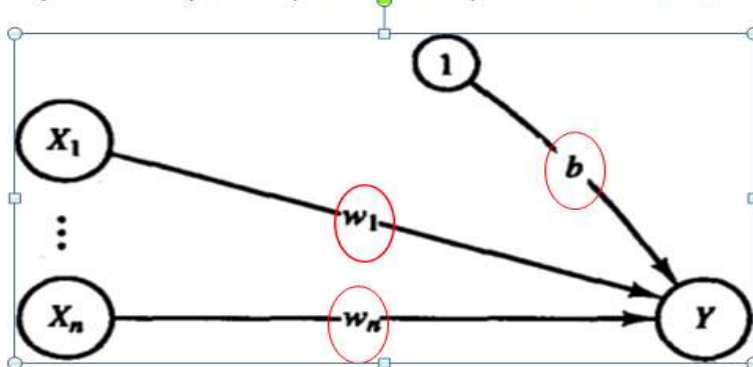
Figure 2.19 Block diagram for explanation of basic learning modes: (a) supervised learning and (b) unsupervised learning.

Adaline

- 1- Architecture
- 2- Algorithm
- 3- Application

ARCHITECTURE

An Adaline is a single unit (neuron) that receives input from several units. It also receives input from a "unit" whose signal is always + 1, in order for the bias weight to be trained by the same process (the delta rule) as is used to train the other weights.



2- Algorithms

- Step 0. Initialize weights.
(Small random values are usually used.)
Set learning rate α .
(See comments following algorithm.)
- Step 1. While stopping condition is false, do Steps 2–6.
- Step 2. For each bipolar training pair $s; t$, do Steps 3–5.
- Step 3. Set activations of input units, $i = 1, \dots, n$:
- $$x_i = s_i.$$
- Step 4. Compute net input to output unit:
- $$y_{in} = b + \sum_i x_i w_i.$$
- Step 5. Update bias and weights, $i = 1, \dots, n$:
- $$b(\text{new}) = b(\text{old}) + \alpha(t - y_{in}).$$
- $$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_{in})x_i.$$
- Step 6. Test for stopping condition:
If the largest weight change that occurred in Step 2 is smaller than a specified tolerance, then stop; otherwise continue.

$$\Delta W1 = x1 * T$$

$$\Delta W2 = x2 * T$$

$$\Delta B = b * T$$

$$E = (T - y_{in})^2$$

3-Application

APPLICATIONS

After training, an Adaline unit can be used to classify input patterns. If the target values are bivalent (binary or bipolar), a step function can be applied as the activation function for the output unit. The following procedure shows the step function for bipolar targets, the most common case:

- Step 0.** Initialize weights
(from Adaline training algorithm given in Section 2.4.2).
- Step 1.** For each bipolar input vector $\mathbf{x}$, do Steps 2–4.
- Step 2.** Set activations of the input units to $\mathbf{x}$.
- Step 3.** Compute net input to output unit:
- $$y_{in} = b + \sum_i x_i w_i.$$
- Step 4.** Apply the activation function:
- $$y = \begin{cases} 1 & \text{if } y_{in} \geq 0; \\ -1 & \text{if } y_{in} < 0. \end{cases}$$

Q1: An Adaline for the OR_GATE bipolar input and target with the initial weight (0.1) and the learning rate (0.1)

X1	X2	B	T	YIN	T-YIN	$\Delta W1$ $\alpha(t-yin)x1$	$\Delta W2$ $\alpha(t-yin)x2$	ΔB $\alpha(t-yin)$	W1 (0.1)	W2 (0.1)	B (0.1)	ERROR E=(T-yin)²
1	1	1	1	0.3	0.7	0.07	0.07	0.07	0.17	0.17	0.17	0.49
1	-1	1	1	0.17	0.83	0.083	-0.083	0.083	0.253	0.087	0.253	0.6889
-1	1	1	1	0.087	0.913	-0.0913	0.0913	0.0913	0.1617	0.1783	0.3443	0.83356
-1	-1	1	-1	0.0043	-1.0043	0.10043	0.10043	-0.10043	0.26213	0.2787	0.2439	1.00861849 E= 3.02107849

Epoch 2

X1	X2	B	T	YIN	T-YIN	$\Delta W1$	$\Delta W2$	ΔB	W1 (0.26213)	W2 (0.2787)	B (0.2439)	ERROR
1	1	1	1	0.78473	0.21557	0.021557	0.021557	0.021557	0.28368	0.3003	0.2654	0.046470
1	-1	1	1	0.2488	0.7512	0.0751	-0.0751	0.0751	0.3588	0.2251	0.3405	0.5643014
-1	1	1	1	0.2069	0.7931	-0.0793	0.0793	0.0793	0.2795	0.3044	0.4198	0.62900761
-1	-1	1	-1	-0.1641	-0.8359	0.0836	0.0836	-0.0836	0.3631	0.388	0.3362	0.69872881 E=1.984977

Epoch 3

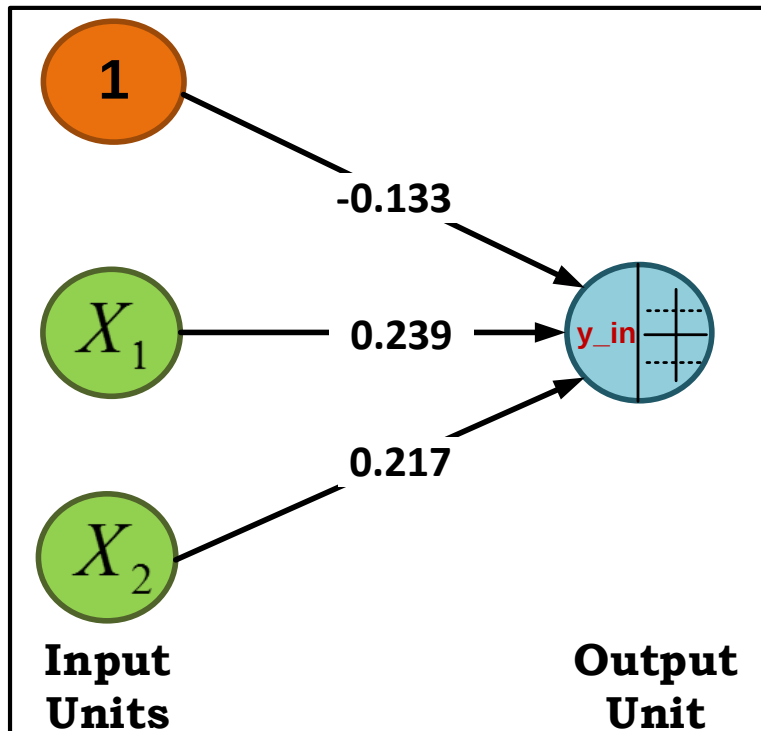
X1	X2	B	T	YIN	T-YIN	$\Delta W1$	$\Delta W2$	ΔB	W1 ()	W2 (0)	B)	ERROR
1	1	1	1	1.0873	0.0873	-0.0087	0.0087	-0.0087	0.3543	0.3793	0.327	0.00762129
1	-1	1	1	0.3025	0.6975	0.0697	-0.0697	0.0697	0.4241	0.3096	0.3973	0.48650625
-1	1	1	1	0.2827	0.7173	-0.0717	0.0717	0.0717	0.3523	0.3813	0.4690	0.51451929
-1	-1	1	-1	- 0.2647	-0.7353	0.0735	0.0735	-0.0735	0.4259	0.4548	0.3954	0.54125999 E=1.54990132

Epoch 4

X1	X2	B	T	YIN	T-YIN	$\Delta W1$	$\Delta W2$	ΔB	W1 ()	W2 ()	B	ERROR
1	1	1	1	1.2761	0.2761	-0.0276	-0.0276	-0.0276	0.17	0.3983	0.3678	0.07623121
1	-1	1	1	0.3389	0.6611	0.0661	-0.0661	0.0661	0.253	0.4644	0.4339	0.43705321
-1	1	1	1	0.3307	0.6693	-0.0669	0.0669	0.0669	0.1617	0.3974	0.5009	0.44796249
-1	-1	1	-1	- 0.3246	-0.6754	0.0675	0.0675	-0.0675	0.26213	0.4650	0.4333	0.45616516 E=1.41741207

Adaline

Epoch 1



Epoch 5

The weights at the end of Epoch 5 is approximately:

$$w1 = 0.5 \quad , \quad w2 = 0.4 \quad , \quad b = -0.3$$

Therefore, the least mean squares (LMS) error is calculated

Inputs			t	<u>y_in</u>	$E = (t - \underline{y_in})^2$
x1	x2	b		w1=0.5, w2=0.4, b= -0.3	
1	1	1	1	0.6	$(1 - 0.6)^2 = 0.16$
1	-1	1	-1	-0.2	$(-1 + 0.2)^2 = 0.64$
-1	1	1	-1	-0.4	$(-1 + 0.4)^2 = 0.36$
-1	-1	1	-1	-1.2	$(-1 + 1.2)^2 = 0.04$

- **Homework**

1- ADALINE net for the ANDNOT function (bipolar inputs and targets)

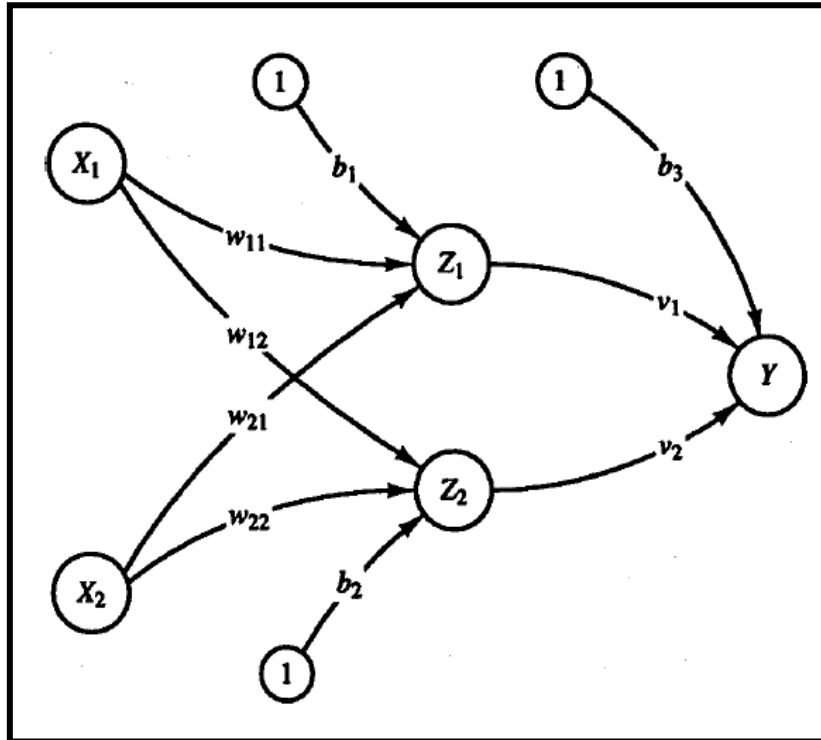
$w_1=w_2=b=0.2$, Learning rate=0.2 (6 Epoch)

2- ADALINE net for the OR function (bipolar inputs and targets)

$w_1=w_2=b=0.1$, Learning rate=0.1 ($E < 1.4$)

Madaline

MADALINE consists of Many Adaptive Linear Neurons (ADALINE) arranged in a multilayer net. A simple MADALINE net is illustrated in bellow Figure



The outputs of the two hidden ADALINES, Z_1 and Z_2 are determined by signals from the same input units X_1 and X_2

As with the Adalines discussed previously, each output signal is the result of applying a threshold function to the unit's net input. Thus, y is a nonlinear function of the input vector (x_1, x_2) .

The use of the hidden units Z_1 and Z_2 , give the net computational capabilities not found in single layer nets, but also complicate the training process.

Training Algorithm for Madaline (MRI). The activation function for units Z_1 , Z_2 , and Y is

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0; \\ -1 & \text{if } x < 0. \end{cases}$$

Step 0. Initialize weights:

Weights v_1 and v_2 and the bias b_3 are set as described;

small random values are usually used for **Adaline** weights.

Set the learning rate a as in the **Adaline** training algorithm (a small value).

Step 1. While stopping condition is false, do Steps 2–8.

Step 2. For each bipolar training pair, $s:t$, do Steps 3–7.

Step 3. Set activations of input units:

$$x_i = s_i.$$

Step 4. Compute net input to each hidden **Adaline** unit:

$$z_{in1} = b_1 + x_1w_{11} + x_2w_{21},$$

$$z_{in2} = b_2 + x_1w_{12} + x_2w_{22}.$$

Step 5. Determine output of each hidden Adaline unit:

$$z_1 = f(z_{in_1}),$$

$$z_2 = f(z_{in_2}).$$

Step 6. Determine output of net:

$$y_{in} = b_3 + z_1v_1 + z_2v_2;$$

$$y = f(y_{in}).$$

Step 7. Determine error and update weights:

If $t = y$, no weight updates are performed.
Otherwise:

If $t = 1$, then update weights on Z_J ,
the unit whose net input is closest to 0,

$$b_J(\text{new}) = b_J(\text{old}) + \alpha(1 - z_{in_J}),$$

$$w_{iJ}(\text{new}) = w_{iJ}(\text{old}) + \alpha(1 - z_{in_J})x_i;$$

If $t = -1$, then update weights on all units Z_k that have positive net input,

$$b_k(\text{new}) = b_k(\text{old}) + \alpha(-1 - z_{in_k}),$$

$$w_{ik}(\text{new}) = w_{ik}(\text{old}) + \alpha(-1 - z_{in_k})x_i.$$

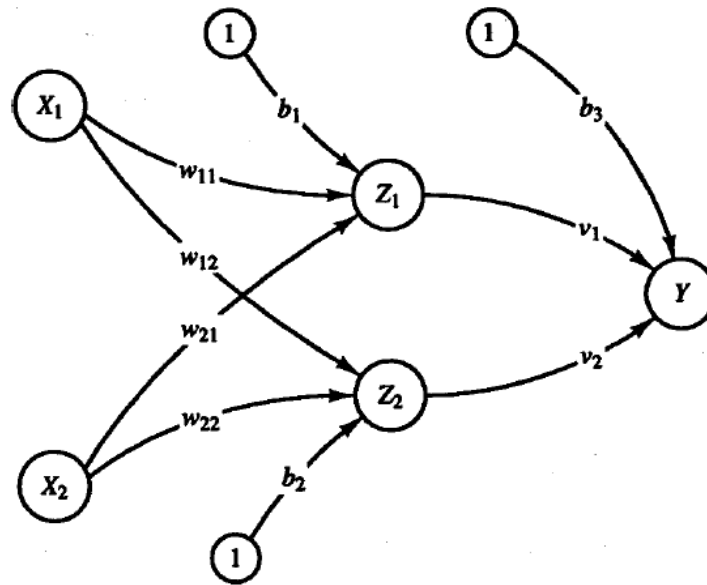
Step 8. Test stopping condition.

If weight changes have stopped (or reached an acceptable level), or if a specified maximum number of weight update iterations (Step 2) have been performed, then stop; otherwise continue.

Example 2.20: Training a MADALINE for the XOR function

This example illustrates the use of the MRI algorithm to train a MADALLNE to solve the XOR problem.

The training patterns are:



Initial values

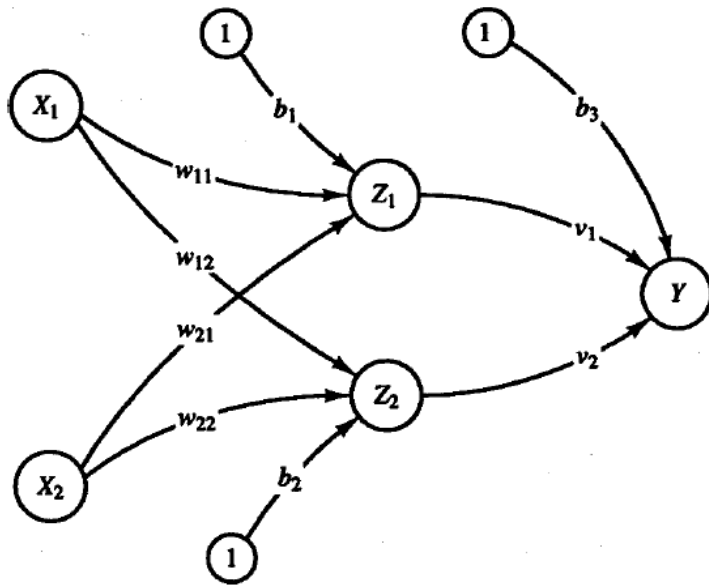
Learning rate $\alpha' = 0.5$

$W_{11} = 0.05, w_{21} = 0.2, b_1 = 0.3$

$W_{12} = 0.1, w_{22} = 0.2, b_2 = 0.5$

$V_1 = v_2 = b_3 = 0.5$

X1	X2	T
1	1	-1
1	-1	1
-1	1	1
-1	-1	-1



The training algorithm is followed.

For first epoch, first input pair (1,1) : 1.

Step 1: Weights and bias are initialized.

$$\begin{aligned}
 v_1 = v_2 = b_3 &= 0.5 \\
 w_{11} = 0.05, w_{21} &= 0.2, b_1 = 0.3 \\
 w_{12} = 0.1, w_{22} &= 0.2, b_2 = 0.15 \\
 \alpha &= 0.5
 \end{aligned}$$

Step 2: Begin training

Step 3: For input and training pair (1,1) : 1, perform steps 4 – 8.

Step 4: $x_i = S_i$

$$x = (1 \ 1)$$

Step 5: Calculate the net input

$$\begin{aligned}
 z_{in1} &= b_1 + x_1 w_{11} + x_2 w_{21} \\
 &= 0.3 + 1 \times 0.05 + 1 \times 0.2 \\
 &= 0.55 \\
 z_{in2} &= b_2 + x_1 w_{12} + x_2 w_{22} \\
 &= 0.15 + 1 \times 0.1 + 1 \times 0.2 \\
 &= 0.45
 \end{aligned}$$

Step 6: Find output z_1 and z_2 apply activations

$$\begin{aligned}
 z_1 &= f(z_{in1}) = 1 \\
 z_2 &= f(z_{in2}) = 1
 \end{aligned}$$

Step 7: Calculate net input of output

$$\begin{aligned}
 y_{in} &= b_3 + v_1 z_1 + v_2 z_2 \\
 &= 0.5 + 1 \times 0.5 + 1 \times 0.5 \\
 &= 1.5
 \end{aligned}$$

Find the output, and apply activations.

$$y = f(y_{in}) = 1$$

Inputs				t	z_{in1}	z_{in2}	z_1	z_2	y_{in}	y	w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2
	x_1	x_2	b													
Epoch 1	1	1	1	-1	0.55	0.45	1	1	1.5	1	-0.725	-0.575	-0.475	-0.625	-0.525	-0.575
	1	-1	1	1	-0.625	-0.675	-1	-1	-0.5	-1	0.0875	-1.3875	0.3375	-0.625	-0.525	-0.575
	-1	1	1	1	-1.1375	-0.475	-1	-1	-0.5	-1	0.0875	-1.3875	0.3375	-1.3625	0.2125	0.1625
	-1	-1	1	-1	1.6375	1.3125	1	1	1.5	1	1.4065	-0.0685	-0.9815	-0.207	1.369	-0.994

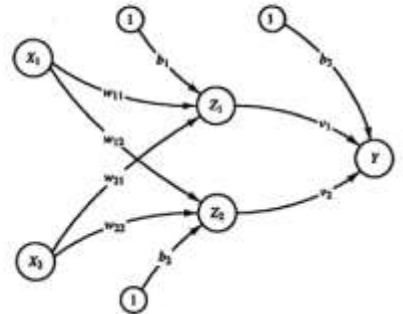
$$w_{11} = 0.05, w_{21} = 0.2, b_1 = 0.3$$

$$w_{12} = 0.1, w_{22} = 0.2, b_2 = 0.5$$

$$v_1 = v_2 = b_3 = 0.5$$

$$z_{in1} = b_1 + x_1 w_{11} + x_2 w_{21}$$

$$z_{in2} = b_2 + x_1 w_{12} + x_2 w_{22}$$



Inputs				t	z_{in1}	z_{in2}	z_1	z_2	y_{in}	y	w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2
	x_1	x_2	b													
Epoch 1	1	1	1	-1	0.55	0.45	1	1	1.5	1	-0.725	-0.575	-0.475	-0.625	-0.525	-0.575
	1	-1	1	1	-0.625	-0.675	-1	-1	-0.5	-1	0.0875	-1.3875	0.3375	-0.625	-0.525	-0.575
	-1	1	1	1	-1.1375	-0.475	-1	-1	-0.5	-1	0.0875	-1.3875	0.3375	-1.3625	0.2125	0.1625
	-1	-1	1	-1	1.6375	1.3125	1	1	1.5	1	1.4065	-0.0685	-0.9815	-0.207	1.369	-0.994

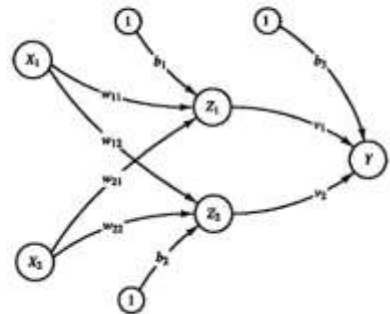
$$w_{11} = 0.05, w_{21} = 0.2, b_1 = 0.3$$

$$w_{12} = 0.1, w_{22} = 0.2, b_2 = 0.5$$

$$v_1 = v_2 = b_3 = 0.5$$

$$z_{in1} = b_1 + x_1 w_{11} + x_2 w_{21}$$

$$z_{in2} = b_2 + x_1 w_{12} + x_2 w_{22}$$



$$w_{11}(\text{new}) = w_{11}(\text{old}) + \alpha(-1 - z_{in1}) \times 1$$

$$= 0.05 + 0.5(-1 - 0.55) \times 1$$

$$= -0.725$$

$$w_{21}(\text{new}) = w_{21}(\text{old}) + \alpha(-1 - z_{in1}) \times 2$$

$$= 0.2 + 0.5(-1 - 0.55) \times 2$$

$$= -0.575$$

$$b_1(\text{new}) = b_1(\text{old}) + \alpha(-1 - z_{in1})$$

$$= 0.3 + 0.5(-1 - 0.55)$$

$$= -0.475$$

$$w_{12}(\text{new}) = w_{12}(\text{old}) + \alpha(-1 - z_{in2}) \times 1$$

$$= 0.1 + 0.5(-1 - 0.45) \times 1$$

$$= -0.625$$

$$w_{22}(\text{new}) = w_{22}(\text{old}) + \alpha(-1 - z_{in2}) \times 2$$

$$= 0.2 + 0.5(-1 - 0.45) \times 2$$

$$= -0.525$$

$$b_2(\text{new}) = b_2(\text{old}) + \alpha(-1 - z_{in2})$$

$$= 0.5 + 0.5(-1 - 0.45)$$

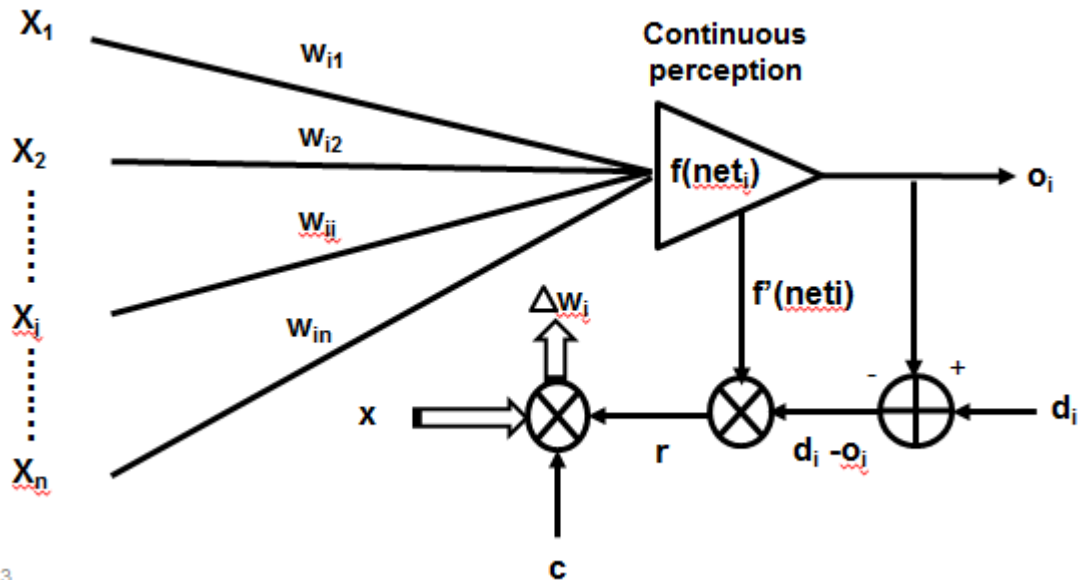
$$= -0.575$$

Inputs				t	z_{in1}	z_{in2}	z_1	z_2	y_{in}	y	w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2
	x_1	x_2	b													
Epoch 2	1	1	1	-1	0.3565	0.168	1	1	1.5	1	0.7285	-0.7465	-1.6595	-0.791	0.785	-1.578
	1	-1	1	1	-0.1845	-3.154	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-0.791	0.785	-1.578
	-1	1	1	1	-3.728	-0.002	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077
	-1	-1	1	-1	-3.728	-1.071	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077

Inputs				t	z_{in1}	z_{in2}	z_1	z_2	y_{in}	y	w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2
	x_1	x_2	b													
Epoch 3	1	1	1	-1	-1.0865	-1.083	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077
	1	-1	1	1	1.5915	-3.655	1	-1	0.5	1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077
	-1	1	1	1	-3.728	1.501	-1	1	0.5	1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077
	-1	-1	1	-1	-1.0495	-1.071	-1	-1	-0.5	-1	1.3205	-1.339	-1.068	-1.292	1.286	-1.077

Delta Learning Rule

Architecture



- ❖ 3 The delta learning rule is only valid for continuous activation functions.
- ❖ It is a supervised training mode.
- ❖ The single weight w is updated using the following increment:

$$\Delta w_i = c(d_i - o_i)f'(net_i)x$$

- ❖ Where $f'(net_i)$ is the derivate of the activation function $f(net_i)$.

Application

Example

$w1 = [1 \ -1 \ 0 \ 0.5]$ $X1 = [1 \ -2 \ 0 \ -1]$ $x2 = [0 \ 1.5 \ -0.5 \ -1]$ $x3 = [-1 \ 1 \ 0.5 \ -1]$

$$\begin{bmatrix} x_1 = \begin{pmatrix} 1 \\ -2 \\ 0 \\ -1 \end{pmatrix} & x_2 = \begin{pmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{pmatrix} & x_3 = \begin{pmatrix} -1 \\ 1 \\ 0.5 \\ -1 \end{pmatrix} & w_1 = \begin{pmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{pmatrix} \end{bmatrix}$$

$c = 0.1$ and $\lambda = 1$. The desired responses for x_1, x_2 , and x_3 are $d_1 = -1, d_2 = -1, d_3 = 1$. Use continues bipolar

activation function $f(net) = \frac{2}{1 + \exp(-\lambda net)} - 1$

$$\begin{aligned} f'(net) &= \frac{(1 + \exp(-\lambda net))(0) - (2)(-\lambda \exp(-\lambda net))}{(1 + \exp(-\lambda net))^2} - 0 \\ &= \frac{2\lambda \exp(-\lambda net)}{(1 + \exp(-\lambda net))^2} \\ &= \frac{\lambda}{2} \left[1 - \left(\frac{2}{1 + \exp(-\lambda net)} - 1 \right)^2 \right] \\ &= \frac{\lambda}{2} (1 - O_i^2) \end{aligned}$$

For $\lambda = 1$

$$F'(net) = \frac{1}{2} (1 - O_i^2)$$

Step 1 :-

Input is x^1 , desired output is d^1 :

$$\text{Net}^1 = w^{1t} * X^1 = [1 \ -1 \ 0 \ 0.5] \begin{pmatrix} 1 \\ -2 \\ 0 \\ -1 \end{pmatrix} = 2.5$$

$$O^1 = f(\text{net}^1) = \frac{2}{1 + e^{-2.5}} - 1 = 0.848 \quad d^1 \neq O^1$$

We thus obtain updated weight vector

$$\begin{aligned} \Delta w^1 &= c (d^1 - O^1) * f'(\text{net}^1) * X^1 \\ &= 0.1 * (-1 - 0.848) (0.140) * \begin{pmatrix} 1 \\ -2 \\ 0 \\ -1 \end{pmatrix} = \begin{pmatrix} -0.0258 \\ 0.0517 \\ 0 \\ 0.0258 \end{pmatrix} \end{aligned}$$

$$\begin{aligned} W^{i+1} &= \Delta w^i + W^i \\ W^2 &= \Delta w^1 + W^1 = \begin{pmatrix} -0.0258 \\ 0.0517 \\ 0 \\ 0.0258 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \\ 0 \\ 0.5 \end{pmatrix} = \begin{pmatrix} 0.974 \\ -0.948 \\ 0 \\ 0.526 \end{pmatrix} \end{aligned}$$

Step 2 :-

Input is x_2 , desired output is d_2 :

$$F(\text{Net}^2) = w^{2t} * X^2 = \begin{pmatrix} 0.974 \\ -0.948 \\ 0 \\ 0.526 \end{pmatrix}^T * \begin{pmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{pmatrix} = -1.948$$

$$O^2 = f(\text{net}^2) = \frac{1}{1 + e^{-1.948}} = -0.750 \quad d_i \neq O_i$$

We thus obtain updated weight vector

$$f'(\text{net}^2) = \frac{1}{2} [1 - (O^2)^2] = \frac{1}{2} [1 - (-0.750)^2] = 0.218$$

$$\Delta w^2 = c (d^2 - O^2) * f'(\text{net}^2) * X^2$$

$$= 0.1 * (-1 + 0.750) * 0.218 * \begin{pmatrix} 0 \\ 1.5 \\ -0.5 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ -8.175 * 10^{-3} \\ 2.725 * 10^{-3} \\ 5.45 * 10^{-3} \end{pmatrix}$$

$$W^{i+1} = \Delta w^i + W^i$$

$$W^3 = \Delta w^2 + W^2 = \begin{pmatrix} 0 \\ -8.175 * 10^{-3} \\ 2.725 * 10^{-3} \\ 5.45 * 10^{-3} \end{pmatrix} + \begin{pmatrix} 0.974 \\ -0.948 \\ 0 \\ 0.526 \end{pmatrix} = \begin{pmatrix} 0.974 \\ -0.956 \\ 0.002 \\ 0.531 \end{pmatrix}$$

Step 3 :-

Input is x^3 , desired output is d^3 :

$$F(\text{Net}^3) = w^{3t} * X^3 = \begin{pmatrix} 0.974 \\ -0.956 \\ 0.002 \\ 0.531 \end{pmatrix}^T * \begin{pmatrix} -1 \\ 1 \\ 0.5 \\ -0.5 \end{pmatrix} = -2.46$$

$$O^3 = f(\text{net}^3) = \frac{2}{1 + e^{-2.46}} - 1 = -0.842 \quad d_i \neq O_i$$

We thus obtain updated weight vector

$$f'(\text{net}^3) = \frac{1}{2} [1 - (O^3)^2] = \frac{1}{2} [1 - (-0.842)^2] = 0.145$$

$$\begin{aligned} \Delta w^3 &= c (d^3 - O^3) * f'(\text{net}^3) * X^3 \\ &= 0.1 * (1 + 0.842) * 0.145 * \begin{pmatrix} -1 \\ 1 \\ 0.5 \\ -0.5 \end{pmatrix} = \begin{pmatrix} -0.026 \\ 0.026 \\ 0.0133 \\ -0.0133 \end{pmatrix} \end{aligned}$$

$$W^{t+1} = \Delta w^t + W^t$$

$$W^4 = \Delta w^3 + W^3 = \begin{pmatrix} -0.026 \\ 0.026 \\ 0.0133 \\ -0.0133 \end{pmatrix} + \begin{pmatrix} 0.974 \\ -0.956 \\ 0.002 \\ 0.531 \end{pmatrix} = \begin{pmatrix} 0.974 \\ -0.929 \\ 0.016 \\ 0.505 \end{pmatrix}$$

Lec. 6

Character Recognition

Ex / classify the two dimensional input pattern (representing letters)
using Hebb. Rule (T, C)

* * *	* * *
*	*
*	* * *
T	C

Solution

Training Pattern

Pattern	Input									Target	
	X1	X2	X3	X4	X5	X6	X7	X8	X9	b	Y
T	1	1	1	-1	1	-1	-1	1	-1	1	1
C	1	1	1	1	-1	-1	1	1	1	1	-1

For the training pair

$$T: X_1 = [1 \ 1 \ 1, -1 \ 1 \ -1, -1 \ 1 \ -1, 1]^T \quad \text{and} \quad Y_1 = [1] \text{ Target}$$

$$W_{\text{new}} = W_{\text{old}} + X_1^T Y_1$$

$$= [0 \ 0 \ 0, 0 \ 0 \ 0, 0 \ 0 \ 0, 0]^T + [1 \ 1 \ 1, -1 \ 1 \ -1, -1 \ 1 \ -1, 1]^T * [1]$$

$$= [1 \ 1 \ 1, -1 \ 1 \ -1, -1 \ 1 \ -1, 1]^T$$

Case 2:

$$C: X_2 = [1 \ 1 \ 1, 1 \ -1 \ -1, 1 \ 1 \ 1, -1]^T \quad \text{and} \quad Y_1 = [-1] \text{ Target}$$

$$W_{\text{new}} = W_{\text{old}} + X_2^T Y_2$$

$$= [1 \ 1 \ 1, -1 \ 1 \ -1, -1 \ 1 \ -1, 1]^T + [1 \ 1 \ 1, 1 \ -1 \ -1, 1 \ 1 \ 1, -1]^T * [-1]$$

$$= [1 \ 1 \ 1, -1 \ 1 \ -1, -1 \ 1 \ -1, 1]^T + [-1 \ -1 \ -1, -1 \ 1 \ 1, -1 \ -1 \ -1, -1]^T$$

$$= [0 \ 0 \ 0, -2 \ 2 \ 0, -2 \ 0 \ -2, 0]$$

$$Y = b + \sum X_i W_i$$

$$= 0 + [1 \ 1 \ 1, -1 \ 1 \ -1, -1 \ 1 \ -1] * [0 \ 0 \ 0, -2 \ 2 \ 0, -2 \ 0 \ -2]$$

$$= 0 + (0 + 0 + 0 + 2 + 2 + 0 + 2 + 0 + 2) = 8 > 0$$

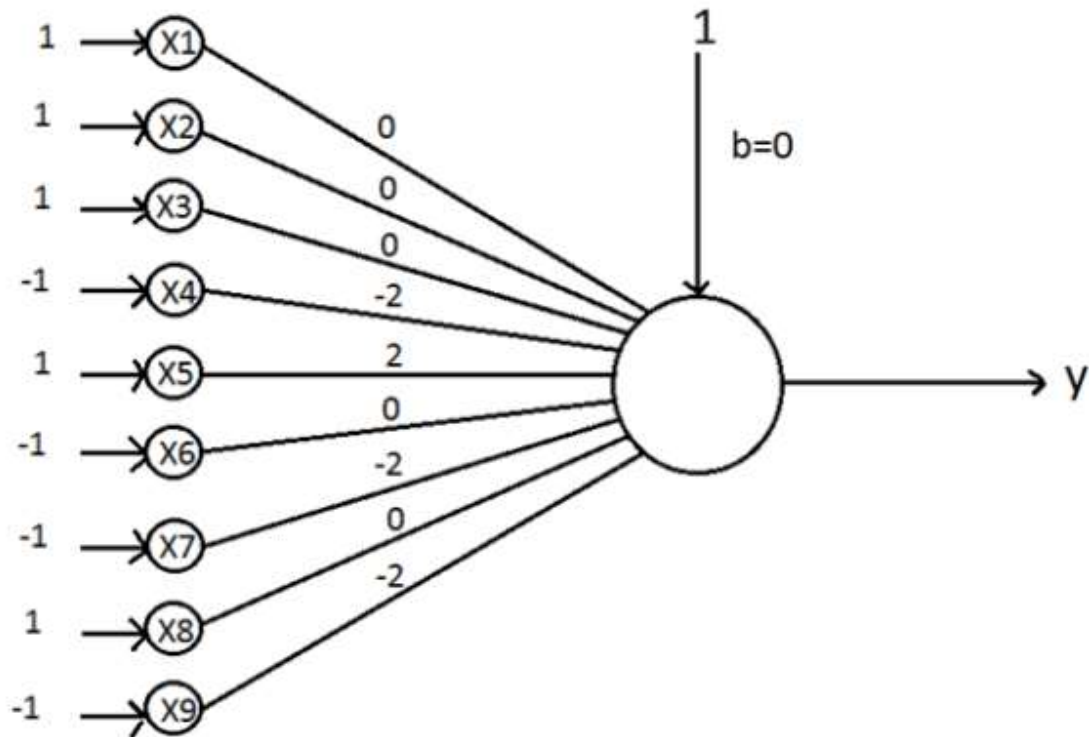
Then the letter is T

$$Y = b + \sum X_i W_i$$

$$= 0 + [1 \ 1 \ 1, 1 \ -1 \ -1, 1 \ 1 \ 1] * [0 \ 0 \ 0, -2 \ 2 \ 0, -2 \ 0 \ -2]$$

$$= 0 + (0 + 0 + 0 - 2 - 2 + 0 - 2 + 0 - 2) = -8 < 0$$

Then the letter is C



Perceptron

2.3 PERCEPTRON

Perceptrons had perhaps the most far-reaching impact of any of the early neural nets. The perceptron learning rule is a more powerful learning rule than the Hebb rule. Under suitable assumptions, its iterative learning procedure can be proved to converge to the correct weights, i.e., the weights that allow the net to produce the correct output value for each of the training input patterns. Not too surprisingly, one of the necessary assumptions is that such weights exist.

A number of different types of perceptrons are described in Rosenblatt (1962) and in Minsky and Papert (1969, 1988). Although some perceptrons were self-organizing, most were trained. Typically, the original perceptrons had three layers of neurons—sensory units, associator units, and a response unit—forming an approximate model of a retina. One particular simple perceptron [Block, 1962] used binary activations for the sensory and associator units and an activation of +1, 0, or -1 for the response unit. The sensory units were connected to the associator units by connections with fixed weights having values of +1, 0, or -1, assigned at random.

The activation function for each associator unit was the binary step function with an arbitrary, but fixed, threshold. Thus, the signal sent from the associator units to the output unit was a binary (0 or 1) signal. The output of the perceptron is $y = f(y_{in})$, where the activation function is

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta_+ \\ 0 & \text{if } -\theta_- \leq y_{in} \leq \theta_+ \\ -1 & \text{if } y_{in} < -\theta_- \end{cases}$$

The weights from the associator units to the response (or output) unit were adjusted by the perceptron learning rule. For each training input, the net would calculate the response of the output unit. Then the net would determine whether an error occurred for this pattern (by comparing the calculated output with the target value). The net did not distinguish between an error in which the calculated output was zero and the target -1, as opposed to an error in which the calculated output was +1 and the target -1. In either of these cases, the sign of the error denotes that the weights should be changed in the direction indicated by the target value. However, only the weights on the connections from units that sent a non-zero signal to the output unit would be adjusted (since only these signals contributed to the error). If an error occurred for a particular training input pattern, the weights would be changed according to the formula

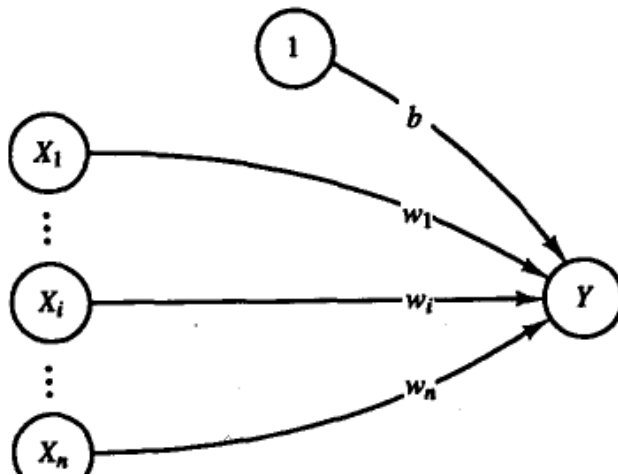
$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i,$$

where the target value t is +1 or -1 and α is the learning rate. If an error did not occur, the weights would not be changed. .

2.3.1 Architecture

Simple perceptron for pattern classification

The output from the associator units in the original simple perceptron was a binary vector; that vector is treated as the input signal to the output unit in the sections that follow. As the proof of the perceptron learning rule convergence theorem given in Section 2.3.4 illustrates, the assumption of a binary input is not necessary. Since only the weights from the associator units to the output unit could be adjusted, we limit our consideration to the single-layer portion of the net, shown in Figure 2.14. Thus, the associator units function like input units, and the architecture is as given in the figure.



2.3.2 Algorithm

The algorithm given here is suitable for either binary or bipolar input vectors (n -tuples), with a bipolar target, fixed θ , and adjustable bias. The threshold θ does not play the same role as in the step function illustrated in Section 2.1.2; thus, both a bias and a threshold are needed. The role of the threshold is discussed following the presentation of the algorithm. The algorithm is not particularly sensitive to the initial values of the weights or the value of the learning rate.

- Step 0.** Initialize weights and bias.
(For simplicity, set weights and bias to zero.)
Set learning rate a ($0 < a \leq 1$).
(For simplicity, a can be set to 1.)
- Step 1.** While stopping condition is false, do Steps 2–6.
- Step 2.** For each training pair $s:t$, do Steps 3–5.
- Step 3.** Set activations of input units:
$$x_i = s_i.$$
- Step 4.** Compute response of output unit:
$$y_{in} = b + \sum_i x_i w_i;$$

$$y = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

Step 5. Update weights and bias if an error occurred for this pattern.

If $y \neq t$,

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i,$$

$$b(\text{new}) = b(\text{old}) + \alpha t.$$

else

$$w_i(\text{new}) = w_i(\text{old}),$$

$$b(\text{new}) = b(\text{old}).$$

Step 6. Test stopping condition:

If no weights changed in Step 2, stop; else, continue.

2.3.3 Application

Logic functions

Example 2.11 A Perceptron for the AND function: binary inputs, bipolar targets

Let us consider again the AND function with binary input and bipolar target, now using the perceptron learning rule. The training data are as given in Example 2.6 for the Hebb rule. An adjustable bias is included, since it is necessary if a single-layer net is to be able to solve this problem. For simplicity, we take $\alpha = 1$ and set the initial weights and bias to 0, as indicated. However, to illustrate the role of the threshold, we take $\theta = 2$.

The weight change is $\Delta w = t(x_1, x_2, 1)$ if an error has occurred and zero otherwise. Presenting the first input, we have:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS		
x_1	x_2	1					w_1	w_2	b
(1	1	1)	0	0	1	(1 1 1)	(0	0	0)
							(1	1	1)

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$					$(w_1 \ w_2 \ b)$
					$(1 \ 1 \ 1)$
$(1 \ 0 \ 1)$	2	1	-1	$(-1 \ 0 \ -1)$	$(0 \ ① \ 0)$

The separating lines become

$$x_2 = .2$$

and

$$x_2 = -.2$$

The graph in Figure 2.16 shows that the response of the net will now (still) be correct for the first input point.

For the third input, we have:

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$					$(w_1 \ w_2 \ b)$
					$(0 \ 1 \ 0)$
$(0 \ 1 \ 1)$	1	1	-1	$(0 \ -1 \ -1)$	$(0 \ 0 \ -1)$

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$					$(w_1 \ w_2 \ b)$
					$(0 \ 0 \ -1)$
$(0 \ 0 \ 1)$	-1	-1	-1	$(0 \ 0 \ -1)$	$(0 \ 0 \ -1)$

The response for all of the input patterns is negative for the weights derived; but since the response for input pattern (1, 1) is not correct, we are not finished.

The second epoch of training yields the following weight updates for the first input:

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$					$(w_1 \ w_2 \ b)$
					$(0 \ 0 \ -1)$
$(1 \ 1 \ 1)$	-1	-1	1	$(1 \ 1 \ 1)$	$(1 \ 1 \ 0)$

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$					$(w_1 \ w_2 \ b)$
					$(1 \ 1 \ 0)$
$(1 \ 0 \ 1)$	1	1	-1	$(-1 \ 0 \ -1)$	$(0 \ 1 \ -1)$

The separating lines become

$$x_2 - 1 = .2$$

and

$$x_2 - 1 = -.2.$$

The graph in Figure 2.18 shows that the response of the net will now be correct (negative) for the input points $(1, 0)$ and $(0, 0)$; the response for input points $(0, 1)$ and $(1, 1)$ will be 0, since the net input would be 0, which is between $-.2$ and $.2$ ($\theta = .2$).

In the second epoch, the third input yields:

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$					$(w_1 \ w_2 \ b)$
					$(0 \ 1 \ -1)$
$(0 \ 1 \ 1)$	0	0	-1	$(0 \ -1 \ -1)$	$(0 \ 0 \ -2)$

INPUT	NET	OUT	TARGET	WEIGHTS CHANGE	WEIGHTS
$(x_1 \ x_2 \ 1)$					$(w_1 \ w_2 \ b)$
					$(0 \ 0 \ -2)$
$(0 \ 0 \ 1)$	-2	-1	-1	$(0 \ 0 \ 0)$	$(0 \ 0 \ -2)$

The results for the third epoch are:

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$					$(w_1 \ w_2 \ b)$
					$(0 \ 0 \ -2)$
$(1 \ 1 \ 1)$	-2	-1	1	$(1 \ 1 \ 1)$	$(1 \ 1 \ -1)$
$(1 \ 0 \ 1)$	0	0	-1	$(-1 \ 0 \ -1)$	$(0 \ 1 \ -2)$
$(0 \ 1 \ 1)$	-1	-1	-1	$(0 \ 0 \ 0)$	$(0 \ 1 \ -2)$
$(0 \ 0 \ 1)$	-2	-1	-1	$(0 \ 0 \ 0)$	$(0 \ 1 \ -2)$

The results for the fourth epoch are:

$(1 \ 1 \ 1)$	-1	-1	1	$(1 \ 1 \ 1)$	$(1 \ 2 \ -1)$
$(1 \ 0 \ 1)$	0	0	-1	$(-1 \ 0 \ -1)$	$(0 \ 2 \ -2)$
$(0 \ 1 \ 1)$	0	0	-1	$(0 \ -1 \ -1)$	$(0 \ 1 \ -3)$
$(0 \ 0 \ 1)$	-3	-1	-1	$(0 \ 0 \ 0)$	$(0 \ 1 \ -3)$

For the fifth epoch, we have

(1	1	1)	-2	-1	1	(1	1	1)	(1	2	-2)
(1	0	1)	-1	-1	-1	(0	0	0)	(1	2	-2)
(0	1	1)	0	0	-1	(0	-1	-1)	(1	1	-3)
(0	0	1)	-3	-1	-1	(0	0	0)	(1	1	-3)

and for the sixth epoch,

(1	1	1)	-1	-1	1	(1	1	1)	(2	2	-2)
(1	0	1)	0	0	-1	(-1	0	-1)	(1	2	-3)
(0	1	1)	-1	-1	-1	(0	0	0)	(1	2	-3)
(0	0	1)	-3	-1	-1	(0	0	0)	(1	2	-3)

The results for the seventh epoch are:

(1	1	1)	0	0	1	(1	1	1)	(2	3	-2)
(1	0	1)	0	0	-1	(-1	0	-1)	(1	3	-3)
(0	1	1)	0	0	-1	(0	-1	-1)	(1	2	-4)
(0	0	1)	-4	-1	-1	(0	0	0)	(1	2	-4)

The eighth epoch yields

(1	1	1)	-1	-1	1	(1	1	1)	(2	3	-3)
(1	0	1)	-1	-1	-1	(0	0	0)	(2	3	-3)
(0	1	1)	0	0	-1	(0	-1	-1)	(2	2	-4)
(0	0	1)	-4	-1	-1	(0	0	0)	(2	2	-4)

and the ninth

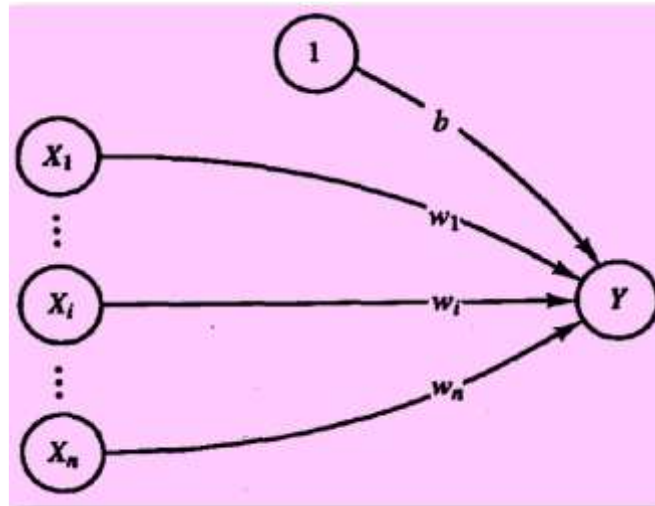
(1	1	1)	0	0	1	(1	1	1)	(3	3	-3)
(1	0	1)	0	0	-1	(-1	0	-1)	(2	3	-4)
(0	1	1)	-1	-1	-1	(0	0	0)	(2	3	-4)
(0	0	1)	-4	-1	-1	(0	0	0)	(2	3	-4)

Finally, the results for the tenth epoch are:

(1	1	1)	1	1	1	(0	0	0)	(2	3	-4)
(1	0	1)	-2	-1	-1	(0	0	0)	(2	3	-4)
(0	1	1)	-1	-1	-1	(0	0	0)	(2	3	-4)
(0	0	1)	-4	-1	-1	(0	0	0)	(2	3	-4)

PERCEPTRON

Architecture



Algorithm

Step 0: Initialize weights and bias .

(For simplicity, set weights and bias to zero.)

Set learning rate a ($0 < a \leq 1$). (For simplicity, a can be set to 1.)

Step 1: While stopping condition is false, do Steps 2-6 .

Step 2: For each training pair $s:t$, do Steps 3-5 .

Step 3: Set activations of input units .

$$X_i = S_i$$

Step 4: Compute response of output unit .

$$Y_{in} = b + \sum_i X_i * w_i$$

$$Y = \begin{cases} 1 & \text{If } Y_{in} > \theta \\ 0 & \text{If } -\theta \leq Y_{in} \leq \theta \\ -1 & \text{If } Y_{in} < -\theta \end{cases}$$

Step 5: Update weights and bias if an error occurred .

If $(Y \neq t) \rightarrow$

$$w_i(\text{new}) = w_i(\text{old}) + a t X_i$$

$$b(\text{new}) = b(\text{old}) + a t$$

else $\rightarrow$

$$w_i(\text{new}) = w_i(\text{old})$$

$$b(\text{new}) = b(\text{old})$$

Step 6: Test stopping .

If no weights changed in Step 2, stop; else: continue.

Application

Example 2.11 :

A Perceptron for the AND function: binary inputs , bipolar targets
take ($a = 1$) and ($\theta = 0.2$) .

Solution :-

❖ The first epoch

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$					$(w_1 \quad w_2 \quad b)$
$(1 \quad 1 \quad 1)$	0	0	1	(1 1 1)	(0 0 0)
$(1 \quad 0 \quad 1)$	2	1	-1	(-1 0 -1)	(0 1 0)
$(0 \quad 1 \quad 1)$	1	1	-1	(0 -1 -1)	(0 0 -1)
$(0 \quad 0 \quad 1)$	-1	-1	-1	(0 0 0)	(0 0 -1)

❖ The second epoch

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$					$(w_1 \quad w_2 \quad b)$
$(1 \quad 1 \quad 1)$	-1	-1	1	(1 1 1)	(0 0 -1)

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$					$(w_1 \quad w_2 \quad b)$
$(1 \quad 0 \quad 1)$	1	1	-1	(-1 0 -1)	(1 1 0)
$(0 \quad 1 \quad 1)$	0	0	-1	(0 -1 -1)	(0 1 -1)

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$					$(w_1 \quad w_2 \quad b)$
$(0 \quad 1 \quad 1)$	0	0	-1	(0 -1 -1)	(0 1 -1)
$(0 \quad 0 \quad 1)$	-2	-1	-1	(0 0 0)	(0 0 -2)

The eighth epoch yields

(1	1	1)	-1	-1	1	(1	1	1)	(2	3	-3)
(1	0	1)	-1	-1	-1	(0	0	0)	(2	3	-3)
(0	1	1)	0	0	-1	(0	-1	-1)	(2	2	-4)
(0	0	1)	-4	-1	-1	(0	0	0)	(2	2	-4)

and the ninth

(1	1	1)	0	0	1	(1	1	1)	(3	3	-3)
(1	0	1)	0	0	-1	(-1	0	-1)	(2	3	-4)
(0	1	1)	-1	-1	-1	(0	0	0)	(2	3	-4)
(0	0	1)	-4	-1	-1	(0	0	0)	(2	3	-4)

Finally, the results for the **tenth epoch** are:

(1	1	1)	1	1	1	(0	0	0)	(2	3	-4)
(1	0	1)	-2	-1	-1	(0	0	0)	(2	3	-4)
(0	1	1)	-1	-1	-1	(0	0	0)	(2	3	-4)
(0	0	1)	-4	-1	-1	(0	0	0)	(2	3	-4)

1. Introduction

Genetic algorithms work on two types of spaces alternatively: coding space and solution space, or in other words, genotype space and phenotype space. Genetic operators (crossover and mutation) work on genotype space, while evolution and selection work on phenotype space. The selection is the link between chromosomes and the performance of decoded solutions. The mapping from genotype space to phenotype space has a considerable influence on the performance of genetic algorithms.

The genetic algorithms provide a directed random search in complex landscapes. There are two important issues with respect to search strategies: exploration (investigate new and unknown areas in search space) and exploitation (make use of knowledge of solutions previously found in search space to help in find better solutions). This can be done by making genetic operators perform essentially a blind search; with a hope that selection operators direct the genetic search toward the desirable area of solution space.

One general principle for developing an implementation of genetic algorithms for a particular real word problem is to make a good balance between exploration and exploitation of the search space. To achieve this, all the operators and parameters of the genetic algorithms must be examined carefully. Addition heuristics should be incorporated in the algorithm to enhance the performance.

2. Genetic Algorithms

Genetic Algorithms (or simply GAs) are powerful and widely applicable stochastic search and optimization methods based on the concepts of natural selection and natural evaluation.

GAs work on a population of individuals represents candidate solutions to the optimization problem. These individual are consists of a strings (called chromosomes) of genes. The genes are a practical allele (gene could be a bit, an integer number, a real value or an alphabet character,...etc depending on the nature of the problem). GAs applying the principles of survival of the fittest , selection , reproduction , crossover (recombining) , and mutation on these individuals to get , hopefully , a new butter individuals (new solutions) .

GAs are applied for those problems which either can not be formulated in exact and accurate mathematical forms and may contain noisy or irregular data or it take so much time to solve or it is simply impossible to solve by the traditional computational methods.

3. How Genetic Algorithms Work

Genetic algorithm maintains a population of individuals, say $\mathbf{P(t)}$, for generation $\mathbf{t}$. Each individual represents a potential solution to the problem at hand. Each individual is evaluated to give some measure of its fitness. Some individuals undergo stochastic transformations by means of genetic operations to form new individuals. There are two type of transformation:-

- 1) Mutation, which creates new individuals by making changes in a single individual.
- 2) Crossover, which creates new individuals by combining parts from two individuals.

The new individuals, called offspring $\mathbf{C(t)}$, are then evaluated. A new population is formed by selecting the more fit individuals from the parent population and offspring population.

After several generations, genetic algorithm converges to the best individual, which hopefully represents an optimal or suboptimal solution to the problem. The general structure of the Genetic algorithms is as follow:

```
Begin
{
  t=0;
  Initialize P(t);
  Evaluate P(t);
  While (not termination condition) do
    Begin
      {
        Apply crossover and mutation to P(t) to yield C(t);
        Evaluate C(t);
        Select P(t+1) from P(t) and C(t);
        t=t+1;
      }
    End
  }
End
```

The flowchart explains how genetic algorithms work is showing in the figure (1).

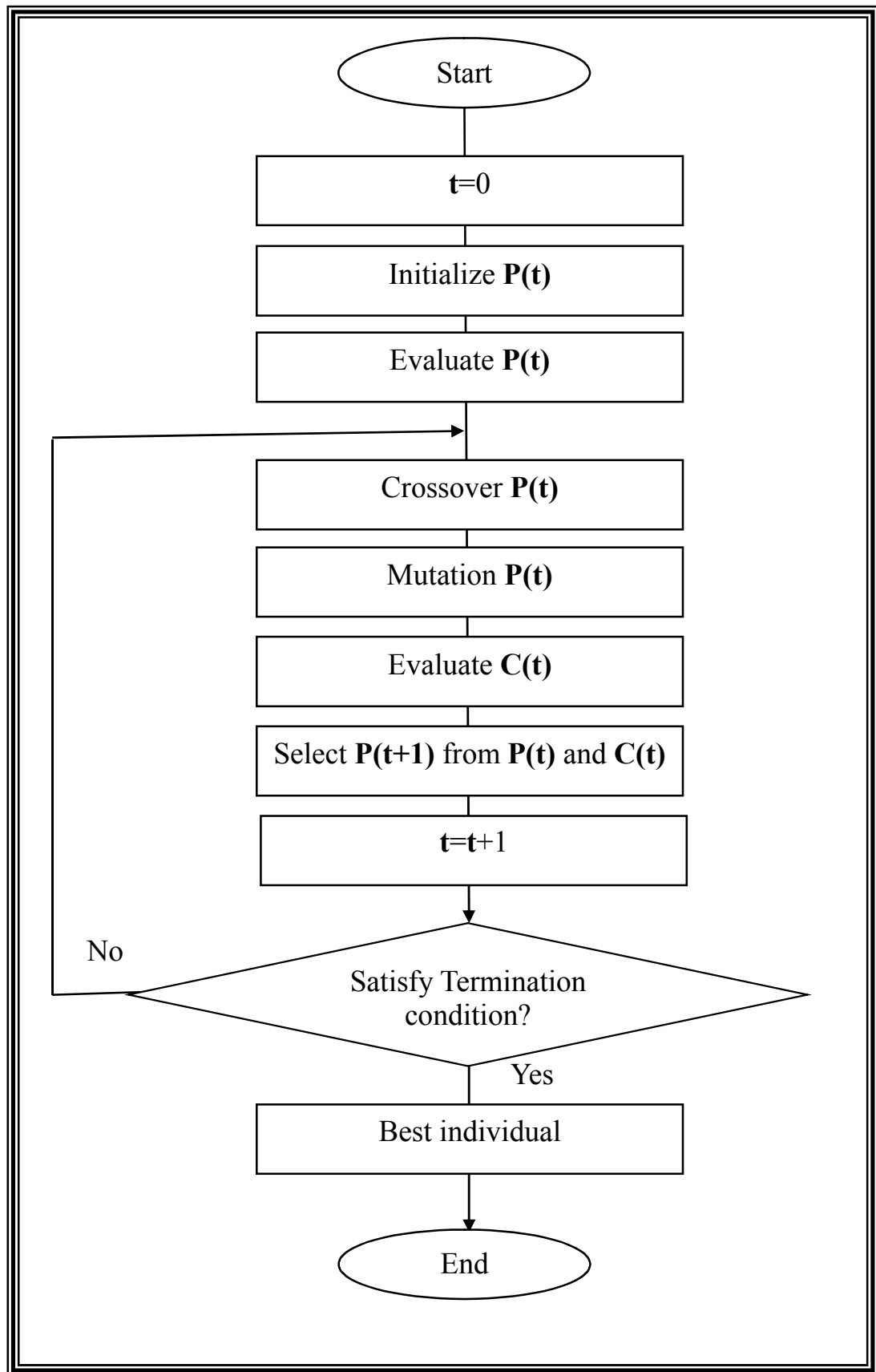


Figure (1) Flowchart explains the general structure of the genetic algorithms.

4. Encoding

How to encode the solutions of the problem into chromosomes is a key issue when using genetic algorithms.

One outstanding problem associated with encoding is that some individuals correspond to **infeasible or illegal solutions** to a given problem. This may become very severe for constrained optimization problems and combinatorial optimization problems.

It must be distinguished between two concepts: **infeasibility** and **illegality**, as shown in figure (2).

Infeasibility refers to the phenomenon that a solution decoded from chromosome lies outside the feasible region of given problem. Penalty methods can be used handle infeasible chromosomes [19]. One of these methods is by force genetic algorithms to approach optimal form both sides of feasible and infeasible regions.

Illegality refers to the phenomenon that a chromosome does not represent a solution to a given problem. Repair techniques are usually adopted to convert an illegal chromosome to legal one.

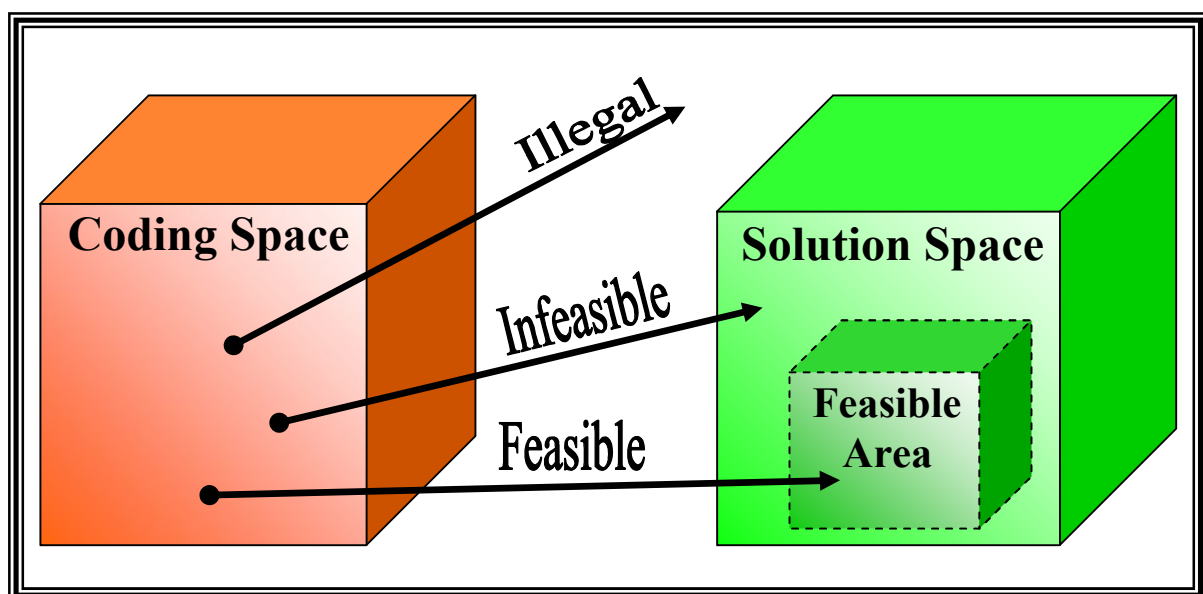


Figure (2) Infeasibility and illegality.

Various encoding methods have been created for particular problems to provide effective implementation of genetic algorithms. According to what kind of symbol is used as the alleles of a gene, the **encoding** methods can be classified as follows:

- 1) Binary encoding
- 2) Real-number encoding
- 3) Integer or literal permutation encoding

4.1 Binary Encoding

Binary encoding (i.e., the bit strings) are the most common encoding used for several of reasons. One is historical: in their earlier work, Holland and his students concentrated on such encodings and genetic algorithms practices have tended to follow this lead. Another reason for that was because much of existing GAs theories is based on the assumption of using binary encoding.

In spite of all that, **binary encoding** for function optimization problems is know to severe drawbacks due to the existence of Hamming cliffs, pairs of encoding having a large Hamming distance (The Hamming distance between two bit strings is defined as the number of corresponding positions in these bit strings where the bits have a different value) while belonging to points of minimal distance in phenotype space. For example, the pair 0111111111 and 1000000000 belongs to neighboring points in phenotype space but have maximum Hamming distance in genotype space. To cross the Hamming cliff, all bits have to be changed simultaneously. The probability that crossover and mutation will occur can be very small. In this sense, the binary code doses not preserve the locality of points in the phenotype space.

For many problems in the industrial engineering world, it is nearly impossible to represent their solution with binary encoding.

4.2 Real Number Encoding

Real number encoding is best used for function optimization problems. It has been widely confirmed that real number encoding perform better than binary encoding for function optimization and constrained optimizations problems. In real number encoding, the structure of genotype space is identical to that of the phenotype. Therefore, it is easy to form effective genetic operators by borrowing useful techniques from conventional methods.

4.3 Integer or Literal Permutation Encoding

Integer or literal permutation encoding is best used for combinational optimization problems because the essence of this kind of problems is to search for the best permutation or combination of items subject to constraints.

5. Genetic Algorithms Operators

There are two basic genetic algorithms operators which are crossover and mutation. These two operators are work together to explore and exploit the search space by creating new variants in the chromosomes. There are many empirical studies on a comparison between crossover and mutation. It is confirmed that mutation operator play the same important role as that of the crossover.

5.1 Crossover

One of the unique aspects of the work involving genetic algorithms (GAs) is the important role that Crossover (recombination) plays in the design and implementation of robust evolutionary systems. In most GAs, individuals are represented by fixed-length strings and crossover operates on pairs of individuals (parents) to produce new strings (offspring) by exchanging segments from the parents' strings. Traditionally, the number of crossover points (which determines how many segments are exchanged) has been fixed at a very low constant value of 1 or 2. Support for this decision came from early work of both a theoretical and empirical nature by Holland. In spite of this, other GAs problems were implemented using other types of crossover.

5.1.1 Single Point Crossover

A commonly used method for crossover is called single point crossover. In this method, a single point crossover position (called cut-point) is chosen at random (e.g., between the 4th and 5th variables) and the parts of two parents after the crossover position are exchanged to form two offspring [1, 24], as shown in figure (3).

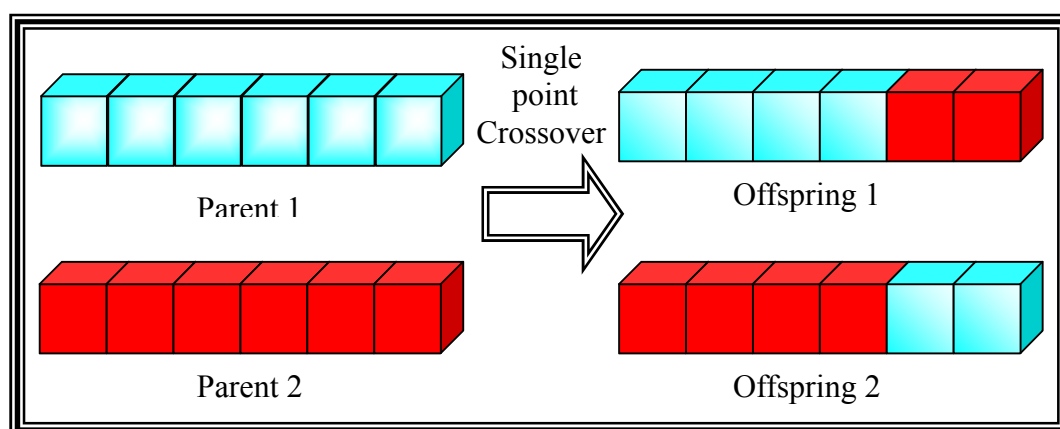


Figure (3) Single point crossover.

5.1.2 Multi Point Crossover

Multi-point crossover is a generalization of single point crossover, introducing a higher number of cut-points. In this case multi positions are chosen at random and the segments between them are exchanged [1, 24], as shown in figure (4).

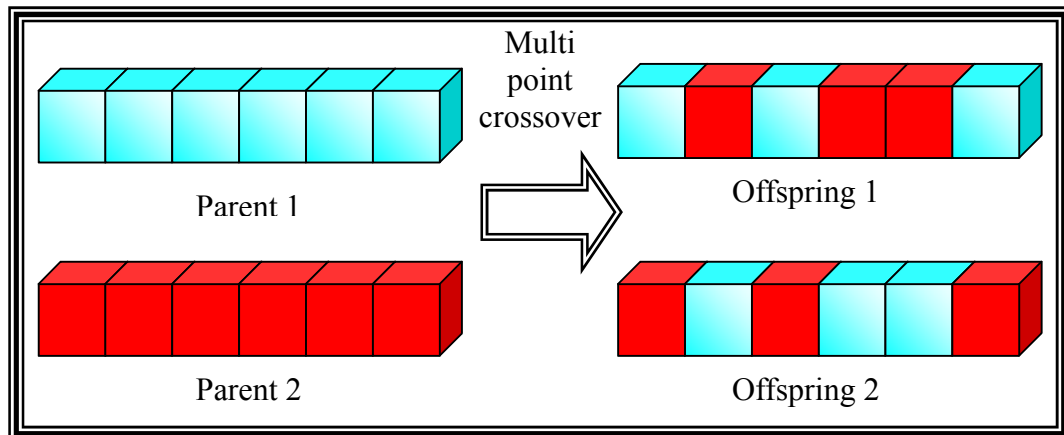


Figure (4) Multi point crossover.

5.1.3 Uniform Crossover

Uniform crossover does not use cut-points, but simply uses a global parameter to indicate the likelihood that each variable should be exchanged between two parents , as shown in figure (5).

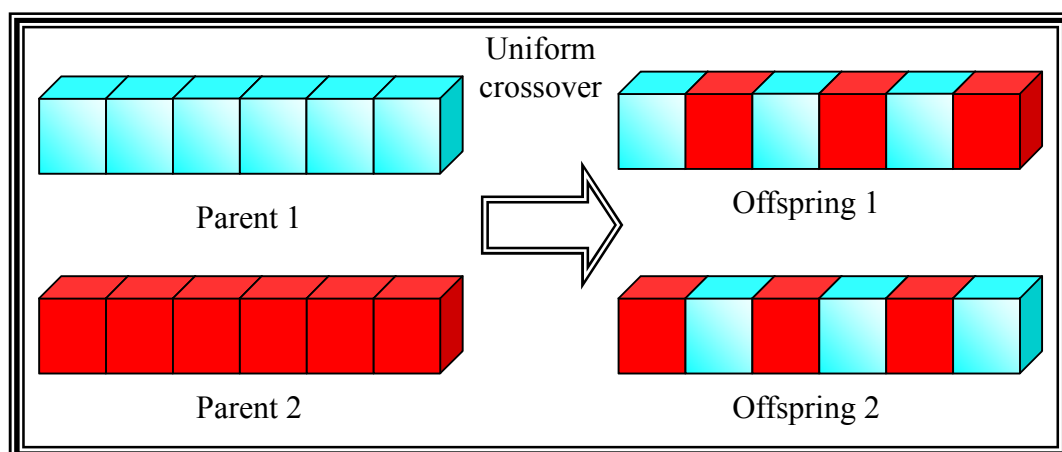


Figure (5) Uniform crossover.

5.2 Mutation

Mutation is a common operator used to help preserve diversity in the population by finding new points in the search space to evaluate. When a chromosome is chosen for mutation, **a random change** is made to the values of **some locations** in the chromosome.

A commonly used method for mutation is called single point mutation. Though, a special mutation types used for varies problem kinds and encoding methods.

5.2.1 Single Point Mutation

Single gene (chromosome or even individual) is randomly selected to be mutated and its value is changed depending on the encoding type used, as shown in figure (6).

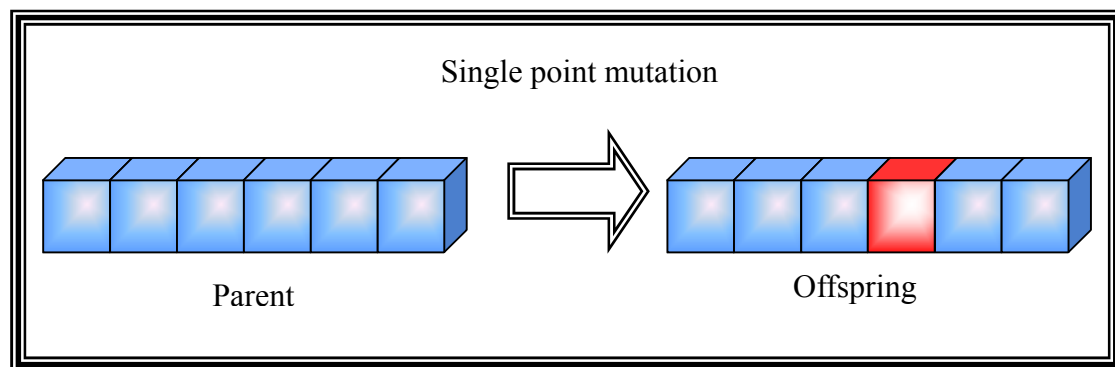


Figure (6) Single point mutation.

5.2.2 Multi Point Mutation

Multi genes (chromosomes or even individuals) are randomly selected to be mutated and there values are changed depending on the encoding type used, as shown in figure (7).

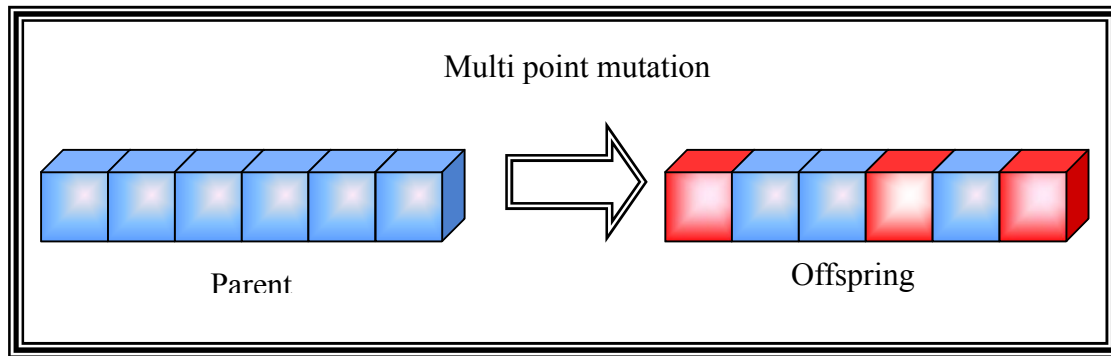


Figure (7) Multi point mutation.

6 .Selection

Selection is the process of determining the number of times a particular individual is chosen for reproduction and, thus, the number of offspring that an individual will produce. The principle behind genetic algorithms is essentially Darwinian natural selection.

Selection provides the driving force in genetic algorithms. With too much force, genetic search will terminate prematurely. While with too little force, evolutionary progress will be slower than necessary.

Typically, a lower selection pressure is indicated at the start of genetic search in favor of a wide exploration of the search space, while a higher selection pressure is recommended at the end to narrow the search space. In this way, the selection directs the genetic search toward promising regions in the search space and that will improve the performance of genetic algorithms. Many selection methods have been proposed, examined and compared. The most common types are :

- 1) Roulette wheel selection
- 2) Rank selection
- 3) Tournament selection
- 4) Steady state selection
- 5) Elitism

6.1 Roulette Wheel Selection

Roulette wheel selection is most common selection method used in genetic algorithms for selecting potentially useful individuals (solutions) for crossover and mutation.

In roulette wheel selection, as in all selection methods, possible solutions are assigned a fitness by the fitness function. This fitness level is used to associate a probability of selection with each individual. While candidate solutions with a higher fitness will be less likely to be eliminated, there is still a chance that they may be. With roulette wheel selection there is a chance some weaker solutions may survive the selection process; this is an advantage, as though a solution may be weak, it may include some component which could prove useful following the recombination process.

The analogy to a roulette wheel can be envisaged by imagining a roulette wheel in which each candidate solution represents a pocket on the wheel; the size of the pockets are proportionate to the probability of selection of the solution. Selecting N individual from the population is equivalent to playing N games on the roulette wheel, as each candidate is drawn independently, as shown in figure (8).

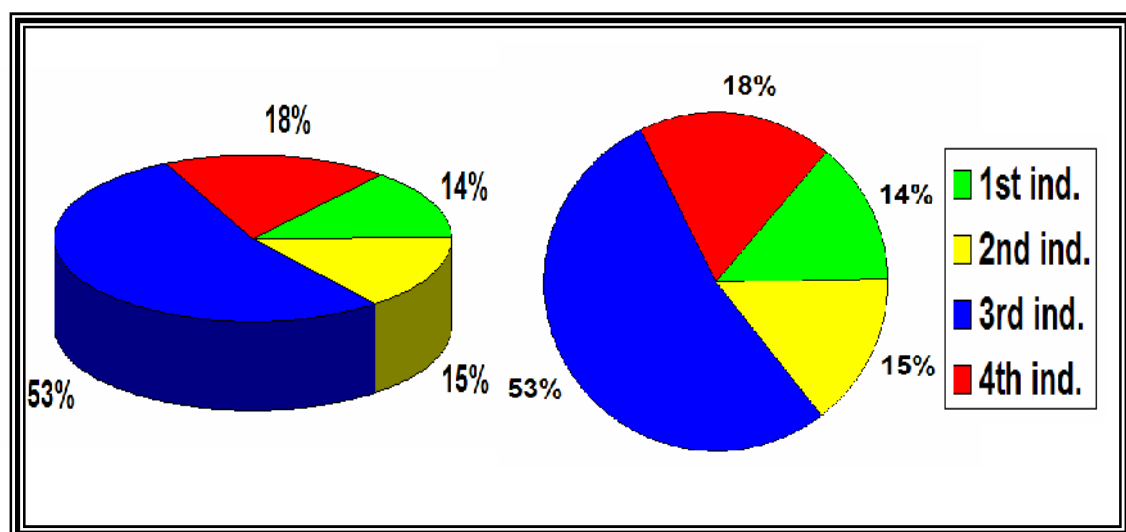
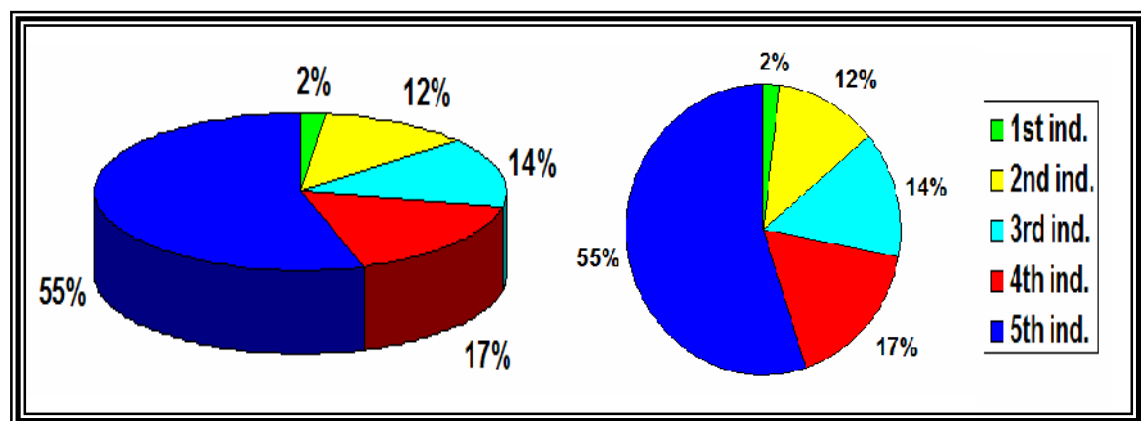


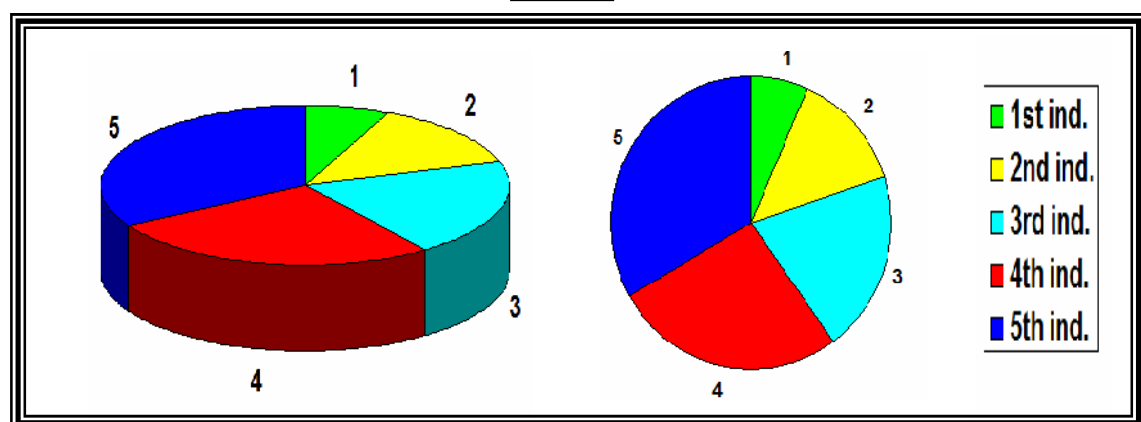
Figure (8) Roulette wheel selection.

6.2 Rank Selection

In ranking selection, the individuals in the population are sorted from best to worst according to their fitness values. Each individual in the population is assigned a numerical rank based on fitness, and selection is based on this ranking rather than differences in fitness. The advantage of this method is that it can prevent very fit individuals from gaining dominance early at the expense of less fit ones, which would reduce the population's genetic diversity and might hinder attempts to find an acceptable solution. The disadvantage of this method is that it required sorting the entire population by rank which is a potentially time consuming procedure. Rank selection effect is shown in figure (9) (a and b).



(a)



(b)

Figure (9) Rank selection effect. (a) Before ranking. (b) After ranking.

6.3 Tournament Selection

This method randomly chooses a set of individual and picks out the best individual for reproduction. The number of individual in the set is called the tournament size. A common tournament size is 2, this is called binary tournament.

By adjusting tournament size, the selection pressure can be made arbitrarily large or small. For example, using large Tournament size has the effect of increasing the selection pressure, since below average individuals are less likely to win a tournament while above average individuals are more likely to win it.

6.4 Steady State Selection

The steady state selection will eliminate the worst of individuals in each generation. It work as follow; the offspring of the individuals selected from each generation go back into the pre-existing population, replacing some of the less fit members of the previous generation .

6.5 Elitism

Elitism is an addition to many selection method that force genetic algorithms to retain some number of the best individual at each generation. It improves the selection process and save the best individuals.

With elitist selection, the quality of the best solution in each generation monotonically increases over time. Without elitist selection, it is possible to lose the best individuals due to stochastic errors (due to crossover, mutation or selection pressure).

7. Genetic Algorithms Parameters

One of the more challenging aspects of using genetic algorithms is to choose the configuration parameter settings. Discussion of GA theory provides little guidance for proper selection of the settings. The population size, the mutation rate, and the type of recombination have the largest effect on search performance. They are used to control the run of a GA. They can influence the Population and the Reproduction part of the GAs. In traditional GAs the parameters has fixed values .

Some guidelines are used in selecting these parameter settings are given in the following subsections.

7.1 Population Size

The population size is one of the most important parameters that plays a significant role in the performance of the genetic algorithms. The population size dictates the number of individuals in the population. Larger population sizes increase the amount of variation present in the initial population at the expense of requiring more fitness evaluations. It is found that the best population size is both application dependent and related to the individual size (number of chromosomes within). A good population of individuals contains a diverse selection of potential building blocks resulting in better exploration .

If the population loses diversity the population is said to have “premature convergence” and little exploration is being done. For larger individuals and challenging optimization problems, larger population sizes are needed to maintain diversity (higher diversity can also be achieved through higher mutation rates and uniform crossover) and hence better exploration. Many researchers suggest population sizes between 25

and 100 individual, while others suggest that it must be very much larger (1000 individual or more).

7.2 Crossover Rate

Crossover rate determines the probability that crossover will occur. The crossover will generate new individuals in the population by combining parts of existing individuals. The crossover rate is usually high and ‘application dependent’. Many researchers suggest crossover rate to be between 0.6 and 0.95 .

7.3 Mutation Rate

Mutation rate determines the probability that a mutation will occur. Mutation is employed to give new information to the population (uncover new chromosomes) and also prevents the population from becoming saturated with similar chromosomes, simply said to avoid premature convergence. Large mutation rates increase the probability that good schemata will be destroyed, but increase population diversity. The best mutation rate is ‘application dependent’. For most applications, mutation rate is between 0.001 and 0.1 [16,36], while for automated circuit design problems, it is usually between 0.3 and 0.8.

8. How Crossover and Mutation Work

Each encoding method have some different in implementation of crossover and mutation. The effect of mutation and crossover on each encoding type will be explained in the following subsections:-

8.1 Binary Encoding

1) Crossover

The single point, multi point, and uniform crossover can implement in binary encoding as explained in section (5.1) before. The effect of each one of these crossover type is shown in the figure (10) (a, b and c).

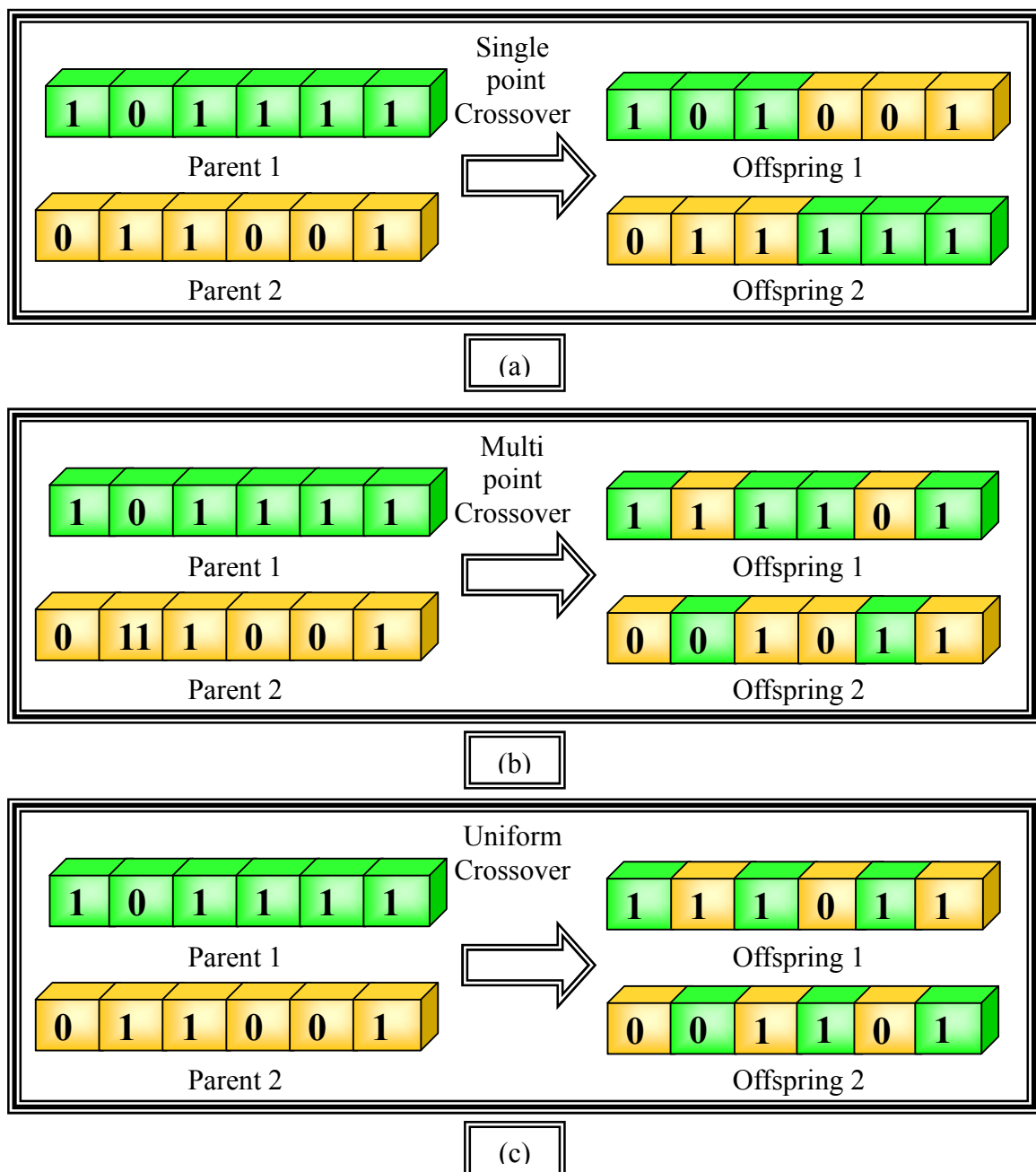
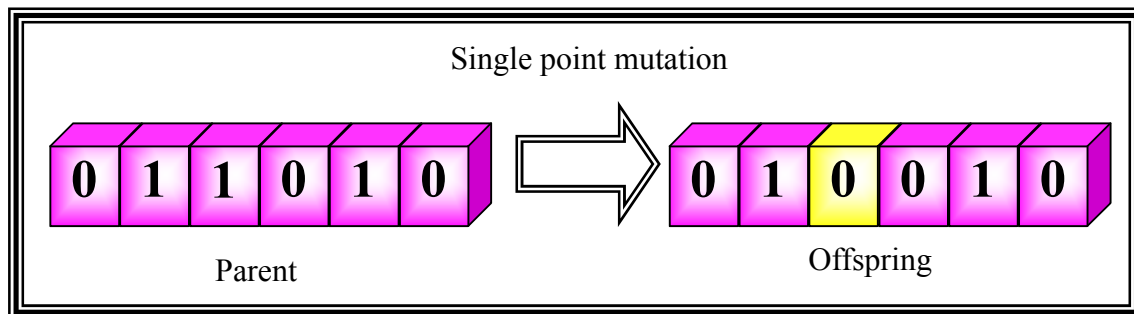


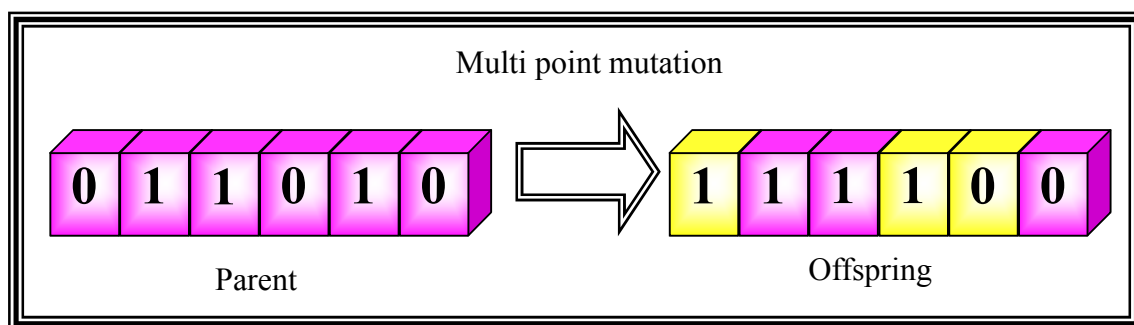
Figure (10) Explanation of crossover effect on binary string. (a) Single point crossover. (b) Multi point crossover. (c) Uniform crossover.

2) Mutation

The single point and multi point can implement in binary encoding as explained in section (5.2) before, the effect of mutation is shown in the figure (11) (a and b).



(a)



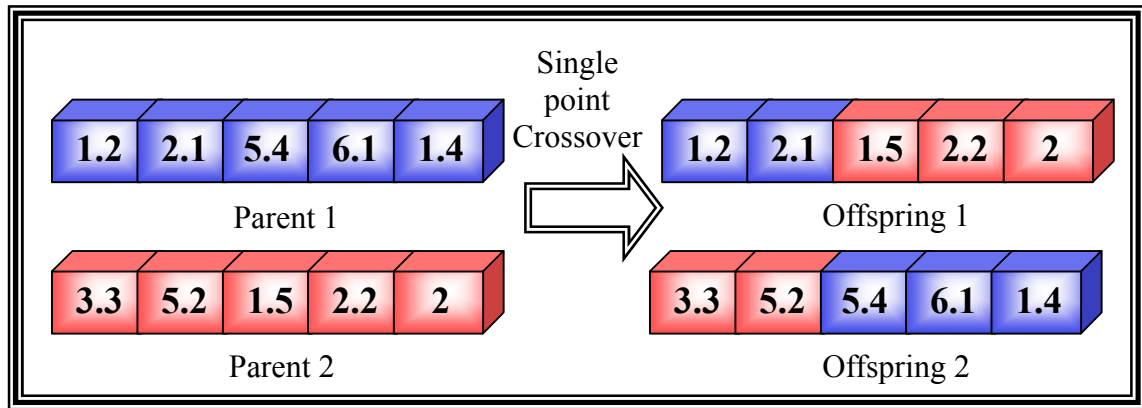
(b)

Figure (11) Explanation of mutation effect on binary string. (a) Single point mutation. (b) Multi point mutation.

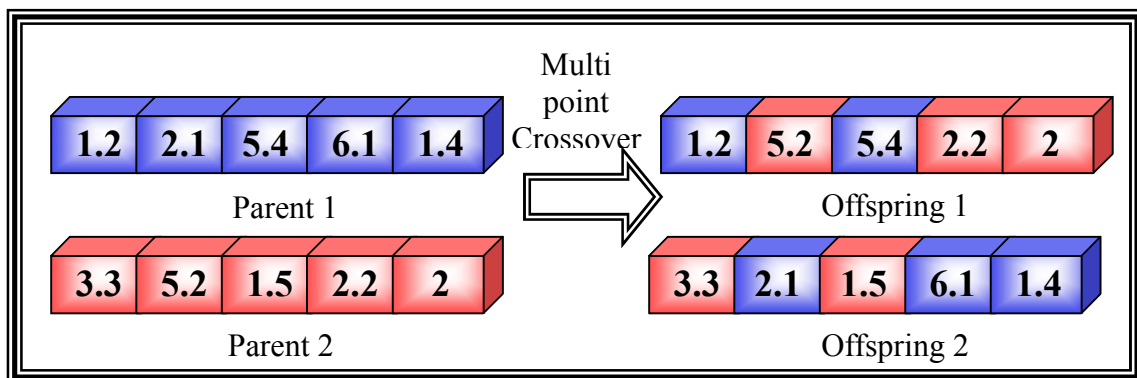
8.2 Real Number Encoding

1) Crossover

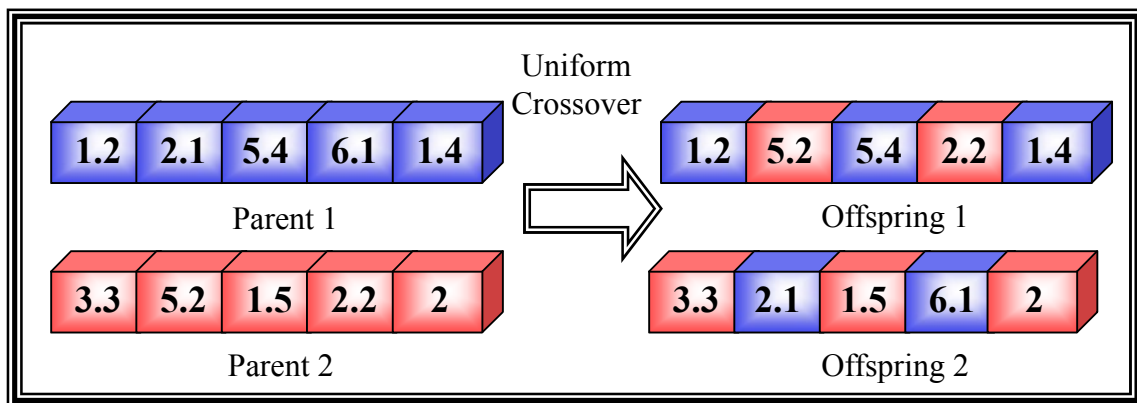
Crossover in real number encoding has the same effect as in binary encoding, the single point, multi point, and uniform crossover in real number encoding is shown in figure (12)(a,b and c).



(a)



(b)



(c)

Figure (12) Explanation of the crossover effect on the real number string.

(a) Single point crossover. (b) Multi point crossover. (c) Uniform crossover.

2) Mutation

In this type of encoding, there is several ways to implement mutation. This done, usually, by adding (or subtract) a random number to (or from) the mutated gene, but in another cases gene might by replaced by a new value generated randomly within the used real number limitation, as shown in figure (13) (a and b).

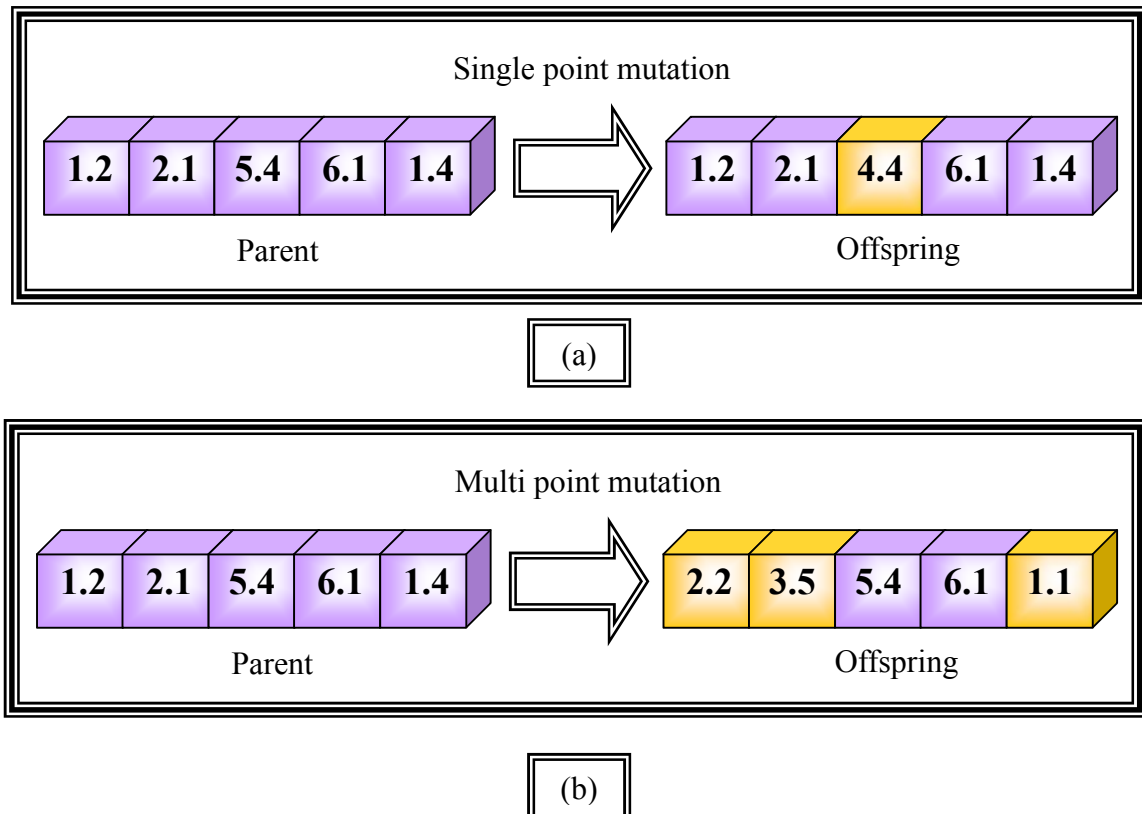


Figure (13) Explanation of the mutation effect on the real number string.

(a) Single point mutation. (b) Multi point mutation.

8.3 Integer or Literal Permutation Encoding

1) Crossover

Unlike the binary and real value encoding, this type has a special crossover rule, for single point crossover the crossover is done as follow: first copy a substring from first parent till crossover point to the first offspring, then copy all the symbols that not contents in that substring from the second parent, do the same to second parent to produce the second offspring. The same procedure can be done in multi point

crossover while uniform crossover is not usually used because it gives illegal offspring . The single point crossover is shown in figure (14).

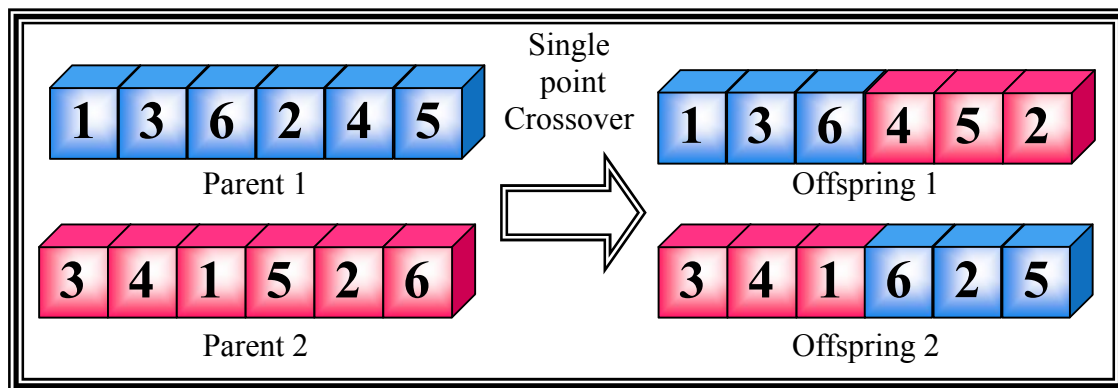
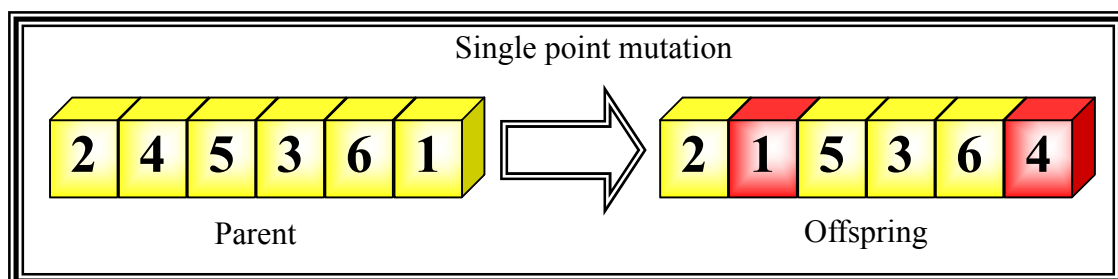


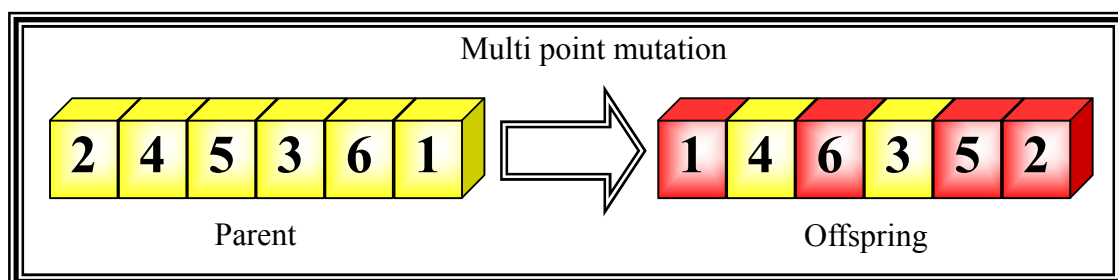
Figure (14) Explanation of single point crossover effect on Integer permutation encoding string.

2) Mutation

A special type of mutation is used, called reorder mutation, in which a pair of genes will select randomly and swap there contents. The mutation effect is shown in figure (15) (a and b).



(a)



(b)

Figure (15) Explanation of the mutation effect on the Integer permutation encoding string. (a) Single point mutation. (b) Multi point mutation.

9. Application of Genetic Algorithms

Genetic algorithms (GAs) are adaptive methods which may be used to solve search and optimization problems. The power of GAs comes from the fact that the technique is robust and can deal successfully with a wide range of problem areas, including those which are difficult for other methods to solve. Therefore, the main ground for GAs is in difficult areas where no such solving techniques exist. Even where existing techniques work well, improvements can be made by mixing them with GAs.

GAs in various forms are implemented to wide rang of problems including the following:

- 1) Optimization: GAs have been used in a wide variety of optimization tasks, including numerical optimization and combinatorial optimization problems such as circuit design and job shop scheduling.
- 2) Automatic Programming: GAs have been used to evolve computer programs for special tasks and to design other computational structures cellular automata and sorting networks.
- 3) Machine and robot learning: GAs have been used for many machine learning applications, including classification and prediction tasks such as the prediction of dynamical systems, weather prediction , and prediction of protein structure. GAs have also been used to design neural networks, and to evolve rules for learning classifier systems or symbolic production systems and to design and control robots
- 4) Economic models: GAs have been used to model processes of innovation, the development of bidding strategies, and the emergence of economic markets.
- 5) Immune system models: GAs have been used to model various aspects of the natural immune system including somatic mutation during an

individual's lifetime and the discovery of multi-gene families during evolutionary time

6) Ecological models: GAs have been used to model ecological phenomena such as biological arms races, host-parasite co-evolution, symbiosis, and resource flow in ecologies.

7) Population genetics models: GAs have been used to study questions in population genetics, such as “under what conditions will a gene for recombination be evolutionarily viable (or practicable) ?”

8) Interactions between evolution and learning: GAs have been used to study how individual learning and species evolution affect one another.

9) Models of social systems: GAs have been used to study evolutionary aspects of social systems, such as the evolution of cooperation, the evolution of communication, and trail-following behavior in ants.

This list is by no means exhaustive, but it gives a flavor of the kinds of things for which GAs have been used, both for problem-solving and for modeling .

10. Advantages and Disadvantage of Genetic Algorithms

Perhaps it isn't obvious why such an algorithm should lead to accurate solutions for optimization problems. Crossover is a crucial aspect of any genetic algorithm, but it may seem that it will dramatically change parents with a high fitness function so that they will no longer be fit. However, this is not the case. As in biology, crossover can lead to new combinations of genes which are more fit than any in the previous generations. Other offspring will be created which are less fit but these will have a low probability of being selected to go on to the next generation. Creating new variants is the key to genetic algorithms, as there is a good chance of finding better solutions. This is why mutation is

also a necessary part of the genetic algorithms. It will create offspring which would not have arisen otherwise, and may lead to a better solution.

Other optimization algorithms have the disadvantage that some kind of initial guess is required and this may bias the final result. GAs on the other hand only require a search range, which need only be constrained by prior knowledge of the physical properties of the system. Effectively they search the whole of the solution space, without calculating the fitness function at every point. This can help avoid a danger in any optimization problem which is being trapped in local maxima or minima. There are two main reasons for this:

- 1) The initial population, being randomly generated, will sample the whole of the solution space, and not just a small area.
- 2) Variation inducing tactics, i.e. crossover and mutation, prevent the algorithm being trapped in one part of the solution space.

Genetic algorithms can be employed for a wide variety of optimization problems. They perform very well for large scale optimization problems which may be very difficult or impossible to solve by other traditional methods.

The disadvantage of genetic algorithms is that it, Sometimes, have trouble finding the exact global optimum because there is no guaranty to find best solution. Another drawback that GAs require large number of response (fitness) function evaluations depending on the number of individuals and the number of generations. Therefore, genetic algorithms may take long time to evaluate the individuals.

Questions and answers ANN

Choose the suitable choice for the following statements

1. The areas in which neural networks are currently being applied are: -----

a. speech Recognition b. searching c. control problems ☒ d. both a&c

2. In neuron input paths called -----

☒ a. dendrites b. synapses c. weights d. axon

3. ----- is the junction between the (axon) of the neuron and the dendrites of the other neuron

☒ a. synapse b. dendrites c. weight d. activation function

4. Information processing occurs at many simple elements called -----

a. weight b. path ☒ c. neuron d. axon

5. Each neuron applies an action -----

6. A Neural network is characterized by: -----

a. Training Learning Algorithm b. Activation function c. architecture ☒ d.

all a,b,c

7. One of ANN properties is

a. serial ☒ b. Fault tolerance c. activation function d. supervision

8. There are ---- types of learning in which the weights organize themselves according to the task to be learnt, these types are-----, -----, -----

a. 3, supervised, unsupervised, activated b. 3, self learned, activated, supervised c. 3, unsupervised, self learned, activated ☒ d. 3, supervised, unsupervised, self learned

9. In the unsupervised learning, the correct final vector is not -----, but instead the ----- are changed through random numbers

a. learned, weight ☒ b. specified, weights c. activated, weights
d. changed, weights

10. In the supervised learning, at every step the system is informed about the -----
- output vector. The ----- are changed according to a formula

- ☒ a. exact, weights b. next, weights c. only, weights d. nearly, weights

11. Neural nets are often classified as

- a. single layer, double layer b. single layer, triple layer ☒ c. single layer, multi layer d. single layer, all layer

12. In determining the number of layers, the input units are ----- as a layer

- ☒ a. not specified ☒ b. not counted c. not determined d. not countered

13. ----- units are between the input units and the output units in the -----
---- net.

- ☒ a. hidden, multi layer b. hidden, double layer c. hidden, triple layer, d.
hidden, single layer

14. the hard limiter activation function is -----& -----

- a. binary, tri-polar bi-polar, tri-polar ☒ c. binary, bi-polar d. binary, single polar

15. the input to binary hard limits is -----& ----- while in the bipolar is -----& -----

- ☒ a. 1, 0, 1,-1 b. 0, 1, 1/2,-1/2 c. 0,1,2,-2 d. 1,-1,0,1

Example2 – Delta learning rule

Perform two training steps of neural network, using the delta learning rule for $\lambda = 1$ and $c = 0.25$. Train the network using the following data pairs

$$\left(\mathbf{x}_1 = \begin{bmatrix} 2 \\ 0 \\ -1 \end{bmatrix}, d_1 = -1 \right), \left(\mathbf{x}_2 = \begin{bmatrix} 1 \\ -2 \\ -1 \end{bmatrix}, d_2 = 1 \right)$$

The initial weights are $\mathbf{W}_1 = [1 \ 0 \ 1]^t$ and $f(\text{net})$ is bipolar continuous activation function.

Solution

$F(\text{net}) = \text{bipolar activation function} =$

$$\frac{2}{1 + \exp(-\lambda x)} - 1$$

$$F'(\text{net}) = \frac{1}{2}[1 - (O_i)^2]$$

$$\text{Net}_1 = \mathbf{X}_1 * \mathbf{W}_1 = [2 \ 0 \ -1] * [1 \ 0 \ 1] = 1$$

$$O_1 = f(\text{net}_1) = 0.47$$

Since $\text{Sgn}(O_1)$ not equal d_1 (-1) the correction is necessary

$$F'(\text{net}_1) = \frac{1}{2}(1 - O^2) = \frac{1}{2}(1 - (0.47)^2) = 0.39$$

$$\mathbf{W}_2 = \mathbf{W}_1 + c(d_i - O_i) F'(\text{net}_1) * \mathbf{X}_1$$

$$\begin{aligned} \mathbf{W}_2 &= [1 \ 0 \ 1] + 0.25(-1 - 0.47) * 0.39 * [2 \ 0 \ -1] = [1 \ 0 \ 1] + (-0.14) * [2 \ 0 \ -1] \\ &= [1 \ 0 \ 1] + [-0.28 \ 0 \ 0.14] \quad \mathbf{W}_2 = [0.72 \ 0 \ 1.14] \end{aligned}$$

$$\text{Net}_2 = \mathbf{X}_2 * \mathbf{W}_2 = [1 \ -2 \ -1] * [0.72 \ 0 \ 1.14] = -0.42$$

$$O_2 = f(\text{net}_2) = -0.2$$

Since $\text{Sgn}(O_2)$ not equal d_2 (1) the correction is necessary

$$F'(\text{net}_2) = \frac{1}{2}(1 - O^2) = \frac{1}{2}(1 - (-0.2)^2) = 0.48$$

$$\mathbf{W}_3 = \mathbf{W}_2 + c(d_2 - O_2) F'(\text{net}_2) * \mathbf{X}_2$$

$$\mathbf{W}_3 = [0.72 \ 0 \ 1.14] + 0.25(1 + 0.2) * 0.48 * [1 \ -2 \ -1]$$

$$\mathbf{W}_3 = [0.72 \ 0 \ 1.14] + [1.44 \ -2.88 \ -1.44]$$

$$\mathbf{W}_3 = [2.16 \ -2.88 \ -0.3]$$

Genetic algorithms

1. GA works on two types of spaces -----& -----

Genotype& phenotype

2. The----- is the link between chromosomes and the performance decoded solutions

Selection

3. GA provides ----- search in complex landscapes

Random

4. There are two important issues with respect to search strategies-----
& -----

Exploration& Exploitation

5. Infeasibility refers to the phenomenon that a solution decoded from chromosome lies----- the feasible region of given problem

outside

6. Genetic operators -----& ----- work on genotype space

Crossover& Mutation

7. Genetic operators -----& ----- work on phenotype space

Evolution& Selection

8. To cross the Hamming cliffs in ----- all bits have to be -----

Binary encoding, changed simultaneously

9. In ----- the structure of genotype space is ----- to that in phenotype.

Real number encoding, Identical

10. Integer or literal permutation encoding is best for -----

Combinational optimization

What are the important issues with respect to search strategies in Genetic Algorithms?

- There are two important issues with respect to search strategies:: exploration (investigate new and unknown areas in search space) and exploitation (make use of knowledge of solutions previously found in search space to help in find better solutions).

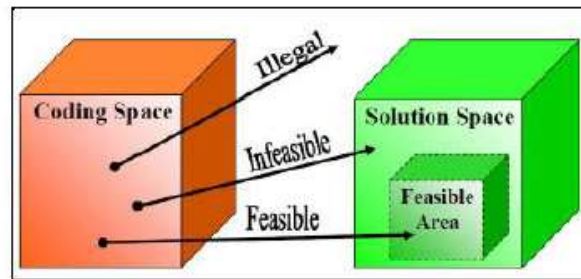
What are Genetic Algorithms? Where GAs work?

- Genetic Algorithms (or simply GAs) are powerful and widely applicable stochastic search and optimization methods based on the concepts of natural selection and natural evaluation.

GAs work on a population of individuals represents candidate solutions to the optimization problem. These individual are consists of a strings (called chromosomes) of genes.

Explain with diagram infeasibility and illegality?

- **Infeasibility** refers to the phenomenon that a solution decoded from chromosome lies outside the feasible region of given problem. Penalty methods can be used handle infeasible chromosomes. One of these methods is by force genetic algorithms to approach optimal form both sides of feasible and infeasible regions.
- **Illegality** refers to the phenomenon that a chromosome does not represent a solution to a given problem. Repair techniques are usually adopted to convert an illegal chromosome to legal one.



Tutorial Sheet of Genetic Algorithm

Part A: Crossover and Mutation

Q₁: Apply single crossover of 6th point and mutation of 3rd point on the following population :

Parent	Pattern	X	2X-1
P1	00000110		
P2	00000010		
P3	00001000		
P4	00001001		
P5	00000110		
P6	00001111		

Solution:

1. Calculate the both value of **X** and **2X-1** from the above Pattern

Parent	Pattern	X	2X-1
P1	00000110	6	11
P2	00000010	2	3
P3	00001000	8	15
P4	00001001	9	17
P5	00000110	6	11
P6	00001111	15	29

2. After crossover 6th point

Children	Pattern	X	2X-1
Child 1	00000010	2	3
Child 2	00000110	6	11
Child 3	00001001	9	17
Child 4	00001000	8	15
Child 5	00000111	7	13
Child 6	00001110	14	27

3. Apply Mutation 3rd point

Children	Pattern	X	2X-1
Child 1	00000010	2	3
Child 2	00000110	6	11
Child 3	00001001	9	17
Child 4	00001000	8	15
Child 5	00000111	7	13
Child 6	00001110	14	27

4. After Mutation 3rd point

Children	Pattern	X	2X-1
Child 1	00100010	34	67
Child 2	00100110	38	75
Child 3	00101001	41	81
Child 4	00101000	40	79
Child 5	00100111	39	77
Child 6	00101110	46	91

Q₂: For the population below with the fitness function $Y = A+B+C+D-30$, apply the following:

1. Single crossover of 4th point
2. Mutation of 4th and 1st points
3. Repeat No 2 with Mutation of 3rd point

Parent	Pattern		
P1	[12;05;07,15]		
P2	[01;08;20,26]		
P3	[05;10;13,20]		
P4	[07;13;09,11]		

Solution:

Calculate the value of **Y**(Evaluation) from the above Pattern

Parent	Pattern	Evaluation $Y = A+B+C+D-30$
P1	[12;05;07,15]	$12+5+7+15-30=9$
P2	[01;08;20,26]	25
P3	[05;10;13,20]	18
P4	[07;13;09,11]	10

1. After crossover 4th point

Children	Pattern	Evaluation $Y = A+B+C+D-30$
Child 1	[12;05;07,26]	20
Child 2	[01;08;20,15]	14
Child 3	[05;10;13,11]	9
Child 4	[07;13;09,20]	19

2. After Mutation of 4th and 1st points

4th in Ch₂

1st in Ch₄

Randomly Ch₂, Ch₄

Children	Pattern	Evaluation $Y = A+B+C+D-30$
Child 1	[12;05;07,26]	20
Child 2	[01;08;20,07]	6
Child 3	[05;10;13,11]	9
Child 4	[15;13;09,20]	27

3. H.W

Part B: Selection: Roulette Wheel Selection

Q₃: Apply roulette wheel selection for the following population:

Parent	Pattern	X	2X-1
P1	00000110		
P2	00000010		
P3	00001000		
P4	00001001		
P5	00000110		
P6	00001111		

Randomly values

1. 0.2
2. 0.01
3. 0.8
4. 0.21
5. 0.11
6. 0.65

Solution:

Calculate the both value of X and 2X-1 from the Pattern

Parent	Pattern	X	2X-1
P1	00000110	6	11
P2	00000010	2	3
P3	00001000	8	15
P4	00001001	9	17
P5	00000110	6	11
P6	00001111	15	29

1. Roulette Selection

Parent	Pattern	X	$F_{obj} = 2X-1$	Fitness Evaluation $F_i = \frac{1}{1+f_{obj}}$	Fitness Probability $P_i = \frac{F_{part}}{F_{all}} = \frac{Fitness}{Total\ Fitness}$		Randomly values	New Population
P1	00000110	6	11	$1/1+11=0.083$	$0.083/0.5544=0.15$	0.15	0.2	P2
P2	00000010	2	3	$1/1+3=0.25$	$0.25/0.5544=0.45$	$0.15+0.45=0.6$	0.01	P1
P3	00001000	8	15	0.0625	0.11	0.71	0.8	P4
P4	00001001	9	17	0.055	0.09	0.8	0.21	P2
P5	00000110	6	11	0.0714	0.13	0.93	0.11	P1
P6	00001111	1	29	0.0333	0.06	$0.98=1$	0.65	P3
				Total Fitness =0.5552				

Hint:

1. Objective function= $f(x) = F_{obj}$
2. Fitness of each individual = $Fitness[i] = (1 / (1+F_{obj}[i]))$
3. Fitness probability $P[i] = Fitness[i] / Total$

Q4: For the following population below, apply the following:

1. Roulette wheel selection
2. Single Crossover over 5th point
3. Mutation of 6th point
4. Repeat No(3) to make **Mutations** of 1st point on **Fourth** pattern and 3rd point on **Five** pattern

Randomly values

1. 0.2
2. 0.01
3. 0.8
4. 0.21
5. 0.11
6. 0.65

Solution:

Calculate the both value of **X** and **2X-1** from the above Pattern

Parent	Pattern	X	2X-1
P1	00000110	6	11
P2	00000010	2	3
P3	00001000	8	15
P4	00001001	9	17
P5	00000110	6	11
P6	00001111	15	29

1. Roulette Selection

Parent	Pattern	X	F_obj= 2X-1	Fitness[]= $\frac{1}{1+F_{obj}}$	Fitness Probability p[]= $\frac{\text{Part}}{\text{All}}$		Randomly values	New Population
P1	00000110	6	11	$1/1+11=0.083$	$0.083/0.5544=0.15$	0.15	0.2	P2
P2	00000010	2	3	$1/1+3=0.25$	$0.25/0.5544=0.45$	$0.15+0.45=0.6$	0.01	P1
P3	00001000	8	15	0.0625	0.11	0.71	0.8	P4
P4	00001001	9	17	0.055	0.09	0.8	0.21	P2
P5	00000110	6	11	0.0714	0.13	0.93	0.11	P1
P6	00001111	15	29	0.0333	0.06	0.98=1	0.65	P3
				Total=0.5552				

Hint:

1. Objective function= $f(x) = F_{obj}$
2. Fitness of each individual = $\text{Fitness}[] = (1 / (1+F_{obj}[]))$
3. Fitness probability $P[i] = \text{Fitness}[] / \text{Total}$

2. Single Crossover over 5th point

New Population

Parent	Pattern	X	2X-1
P1	00000010	2	3
P2	00000110	6	11
P3	00001001	9	17
P4	00000010	2	3
P5	00000110	6	11
P6	00001000	8	15

After Crossover over 5th point

Children	Pattern	X	2X-1
Child 1	00000110	6	11
Child 2	00000010	2	3
Child 3	00000010	2	3
Child 4	00001010	10	19
Child 5	00001000	8	15
Child 6	00000110	6	11

3. After Mutation of 6th point

Children	Pattern	X	2X-1
Child 1	00000010	2	11
Child 2	00000110	6	3
Child 3	00000110	6	3
Child 4	00001110	14	19
Child 5	00001100	12	15
Child 6	00000010	2	11

4. H.W

Q₅: For the following population below with the function $Y = A+B+C+D-30$, apply the following:

1. Roulette wheel selection
2. Single Crossover over 4th point
3. Mutation for 4th and 1st point

Randomly values

1. 0.1
2. 0.7
3. 0.3
4. 0.5

Parent	Pattern		
P1	[12;05;07,15]		
P2	[01;08;20,26]		
P3	[05;10;13,20]		
P4	[07;13;09,11]		

Solution: Calculate the value of **Y**(Evaluation) from the above Pattern

Parent	Pattern	Evaluation $Y = A+B+C+D-30$
P1	[12;05;07,15]	$12+5+7+15-30=9$
P2	[01;08;20,26]	25
P3	[05;10;13,20]	18
P4	[07;13;09,11]	10

1. Roulette Selection

Parent	Pattern	$F_{obj} = Y = A+B+C+D-30$	$Fitness[] = \frac{1}{1+F_{obj}}$	Fitness Probability $p[] = \frac{F_{part}}{F_{all}}$		Randomly values	New population
P1	[12;05;07,15]	9	0.1	0.355	0.355	0.1	P1
P2	[01;08;20,26]	25	0.038	0.135	0.49	0.7	P4
P3	[05;10;13,20]	18	0.053	0.188	0.678	0.3	P1
P4	[07;13;09,11]	10	0.09	0.32	1	0.5	P3
			Total Fitness = 0.2819				

The new population will be:

Parent	Pattern
P1	[12;05;07,15]
P2	[07;13;09,11]
P3	[12;05;07,15]
P4	[05;10;13,20]

Hint:

1. Objective function= $f(x) = F_{obj}$
2. Fitness of each individual = $Fitness[] = (1 / (1+F_{obj}[]))$
3. Fitness probability $P[i] = Fitness[] / Total$

2. After Single Crossover 4th point

Children	Pattern
Child 1	[12;05;07,11]
Child 2	[07;13;09,15]
Child 3	[12;05;07, 20]
Child 4	[05;10;13,15]

3. Mutation for 4th and 1st point

By assuming the Mutation Rate= Population size $\times$ rate; rate=0.4 (Given)

$$= 4 \times 0.4 = 2$$

Choose randomly c[2], c[4]

Point 4,1

Children	Pattern
Child 1	[12;05;07,11]
Child 2	[07;13;09,15]
Child 3	[12;05;07, 20]
Child 4	[05;10;13,15]

After Mutation

Children	Pattern
Child 1	[12;05;07,11]
Child 2	[07;13;09,05]
Child 3	[12;05;07, 20]
Child 4	[15;10;13,15]

The new population can be represented as Roulette Selection (H.W)

Parent	Pattern	$F_{obj} = Y = A+B+C+D-30$	$Fitness[] = \frac{1}{1+f_{obj}}$	$Fitness\ Probability\ p[] = \frac{Part}{All}$	Randomly	New population
P1	[12;05;07,11]				0.1	P1
P2	[07;13;09,05]				0.7	P4
P3	[12;05;07,20]				0.3	P1
P4	[15;10;03,15]				0.5	P3
			Total=			

Hint:

1. Objective function= $f(x) = F_{obj}$
2. Fitness of each individual = $Fitness[] = (1 / (1+F_{obj}[]))$
3. Fitness probability $P[i] = Fitness[] / Total$

Q₆:H.W: Suppose a genetic algorithm uses populations as in pattern below with fitness function of individual x: $f(x) = a + 2b + 3c + 4d$

$$x_1 = [a;b;c;d] = [12;05;23;08]$$

$$x_2 = [a;b;c;d] = [02;21;18;03]$$

$$x_3 = [a;b;c;d] = [10;04;13;14]$$

$$x_4 = [a;b;c;d] = [20;01;10;06]$$

1. Calculate the objective function of each individual
2. Compute the fitness of each individual
3. Calculate the fitness probability of each individual
4. Apply Single Crossover of 2nd point
5. Apply Mutation for 1st and 3rd point

Hint:

1. Objective function= $f(x) = F_{obj}$
2. Fitness of each individual = $Fitness[] = (1 / (1+F_{obj}[]))$
3. Fitness probability $P[i] = Fitness[] / Total$

Part C: Questions and Answers

Q₁: What are the important issues with respect to search strategies in Genetic Algorithms?

Answer:

There are two important issues with respect to search strategies:

1. Exploration (investigate new and unknown areas in search space)
2. Exploitation (make use of knowledge of solutions previously found in search space).

Q₂: What are Genetic Algorithms? Where GAs work?

Answer:

Genetic Algorithms (or simply GAs) are powerful and widely applicable stochastic search and optimization methods based on the concepts of natural selection and natural evaluation.

GAs work on a population of individuals represents candidate solutions to the optimization problem.

Q₃: State the Encoding methods? Explain each.

Answer:

Encoding methods can be classified as follows:

- 1) Binary encoding
- 2) Real-number encoding
- 3) Integer or literal permutation encoding

Q₄: What are the two requirements should a problem satisfy in order to be suitable for solving it by a GA?

Answer:

1. The fitness function can be well-defined.
2. Solutions should be decomposable into steps (building blocks) which could be then encoded as chromosomes.

Q₆: What is the general structure of the Genetic Algorithms?

Answer:

The general structure of the Genetic algorithms is as follow:

Begin

{

t=0;

Initialize P(t);

Evaluate P(t);

While (not termination condition) do

Begin

{

Apply crossover and mutation to P(t) to yield C(t);

Evaluate C(t);

Select P(t+1) from P(t) and C(t);

t=t+1;

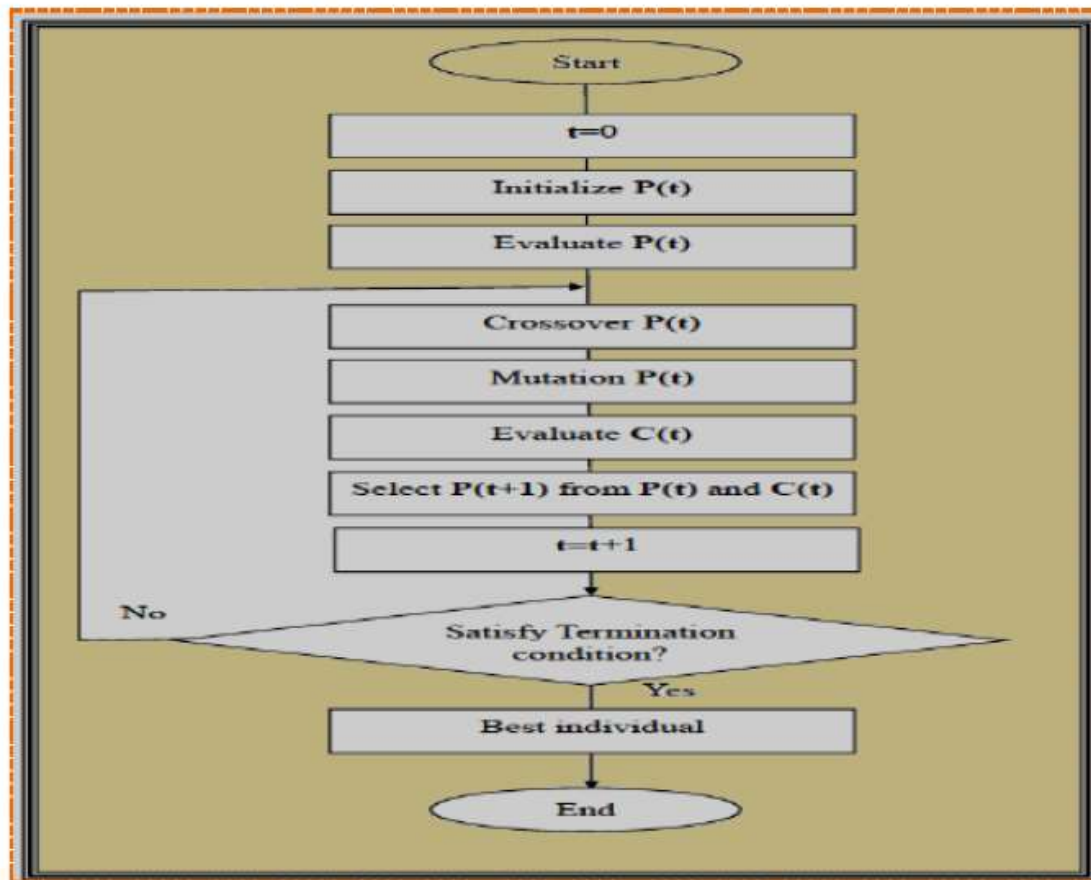
}

End

}

End

Q7. Based on the flowchart explains how genetic algorithms work?



Q₈: List Six Applications of Genetic Algorithms

Answer:

1. Optimization
2. Automatic Programming
3. Machine and robot learning
4. Economic models
5. Immune system models
6. Population genetics models

Q₉: Mention the most Advantages and Limitations of Genetic Algorithm

The advantages of genetic algorithm includes,

1. Parallelism
2. Liability
3. Solution space is wider
4. The fitness landscape is complex
5. Easy to discover global optimum

The limitation of genetic algorithm includes,

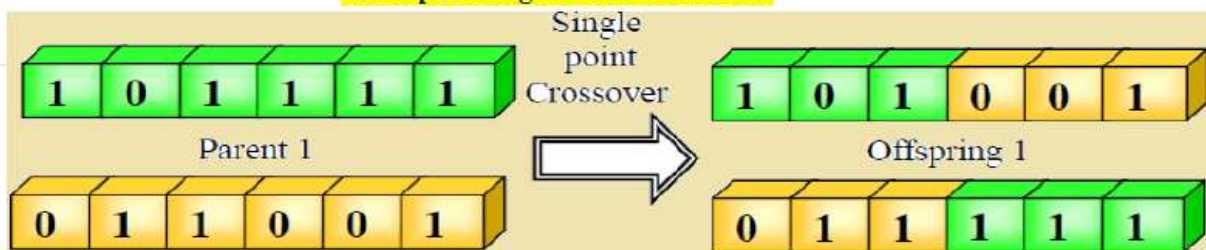
1. The problem of identifying fitness function
2. Definition of representation for the problem
3. Premature convergence occurs
4. The problem of choosing the various parameters like the size of the population, Mutation rate, cross over rate, the selection method and its strength.
5. Cannot use gradients

Q₁₀: State the types of crossover and give **only one** example for any type.

Answer:

1. **Single Point Crossover**
2. **Multi Point Crossover**
3. **Uniform Crossover**

Example: Single Point Crossover

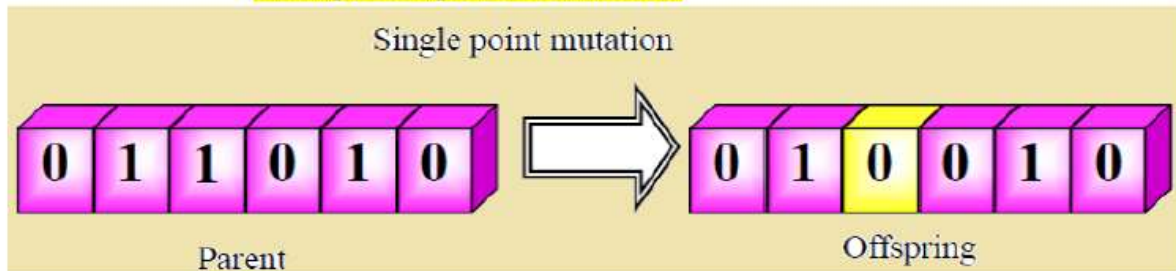


Q₁₁: State the types of mutation and give **only one** example for any type

Answer:

1. **Single Point Mutation**
2. **Multi Point Mutation**

Example: Single Point Mutation



Q₁₂: Choose the suitable words for the following point:

1. GA works on two types of spaces &
2. The is the link between chromosomes and the performance decoded solutions.
3. GA provides search in complex landscapes

4. There are two important issues with respect to search strategies
..... &
5. Infeasibility refers to the phenomenon that a solution decoded from
chromosome lies the feasible region of given problem
6. Genetic operators&work on genotype space
7. To cross the Hamming cliffs in all bits have to be
.....
8. Genetic operators&work on phenotype space
9. In the structure of genotype space is to that in
phenotype
10. Integer or literal permutation encoding is best for